WeMust AI API 使用指南

目錄

- WeMust AI API 使用指南
  - 目錄
  - 第一章：快速開始指南
    - 1.1 什麼是WeMust AI API
    - 1.2 準備工作（獲取API密鑰）
    - 1.3 第一次API調用（5分鐘實戰）
    - 1.4 我想要簡單直接的使用AI作爲各式助手：Cherry Studio
    - 1.6 Dify配置
    - 1.5 常見問題快速解決
  - 第二章：API基礎知識
    - 2.1 API概述和架構
    - 2.2 認證機制詳解
      - 2.2.1 API密鑰認證
    - 2.3 基礎URL和端點
    - 2.4 支持的AI模型介紹
  - 第三章：核心功能使用
    - 3.1 聊天對話功能
      - 3.1.1 基本對話實現
      - 3.1.2 系統角色設定
      - 3.1.3 對話歷史管理
    - 3.2 流式響應處理
      - 3.2.1 什麼是流式響應
      - 3.2.2 實現實時對話效果
    - 3.3 工具調用功能
      - 3.3.1 工具調用概念
      - 3.3.2 自定義函數集成
  - 第四章：參數調優指南
    - 4.1 核心參數詳解
      - 4.1.1 temperature（創意度控制）
      - 4.1.2 max_tokens（回覆長度控制）
      - 4.1.3 top_p（多樣性控制）
    - 4.2 不同場景的參數配置
      - 4.2.1 學術問答場景
      - 4.2.2 創意寫作場景
      - 4.2.3 代碼生成場景
    - 4.3 參數調優實戰練習
  - 第五章：編程實戰教程
    - 5.1 Python開發環境設置
    - 5.2 基礎聊天機器人開發
      - 5.2.1 命令行版本
      - 5.2.2 Web版本

        - 5.3 進階應用開發
            - 5.3.1 智能問答系統
    - 第六章：完整API參考
        - 6.1 請求格式規範
        - 6.2 響應格式說明
        - 6.3 錯誤碼完整列表
        - 6.4 模型能力對比表
    - 附錄
        - A. 常見問題解答（FAQ）
            - A.1 認證和API密鑰相關
            - A.2 技術使用相關
            - A.3 開發相關
            - A.4 錯誤處理相關
    - 結語

第一章：快速開始指南

1.1 什麼是WeMust AI API

WeMust AI API是澳門科技大學提供的人工智能服務接口，讓你能夠輕鬆地將先進的AI功能集成到自己的應用程序中。通過這個API，你可以訪問多種強大的AI模型，包括GPT-4o、GLM-4等，實現智能對話、內容生成、問答系統等功能。

想像一下，你可以用幾行代碼就讓你的程序擁有像ChatGPT一樣的對話能力，或者讓它幫你寫文章、回答問題、翻譯語言。這就是WeMust AI API能為你帶來的強大功能。

主要特點：

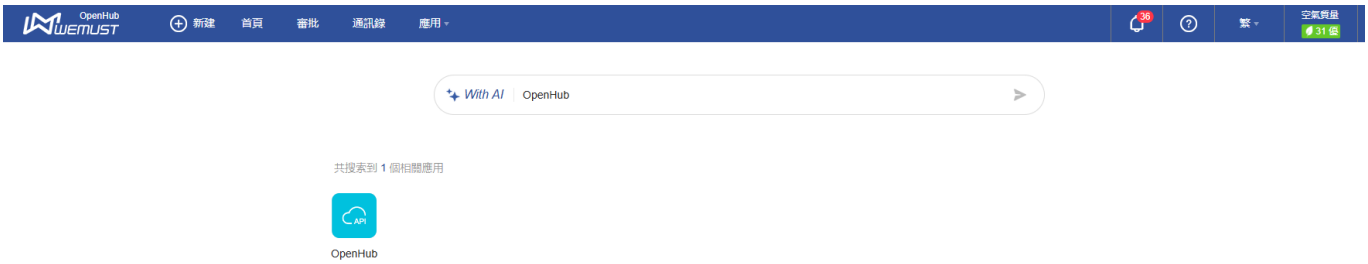
- 支持多種先進的AI模型
- 簡單易用的RESTful API接口
- 支持實時流式響應
- 提供工具調用功能
- 完善的錯誤處理機制

1.2 準備工作（獲取API密鑰）

在開始使用API之前，你需要獲取一個API密鑰。這就像是你進入AI世界的鑰匙，每次調用API時都需要用它來證明你的身份。

步驟1：註冊賬號

1. 訪問WeMust AI平台：`https://openhub-web.must.edu.mo/api-apply` / 或在WeMust中搜索OpenHub

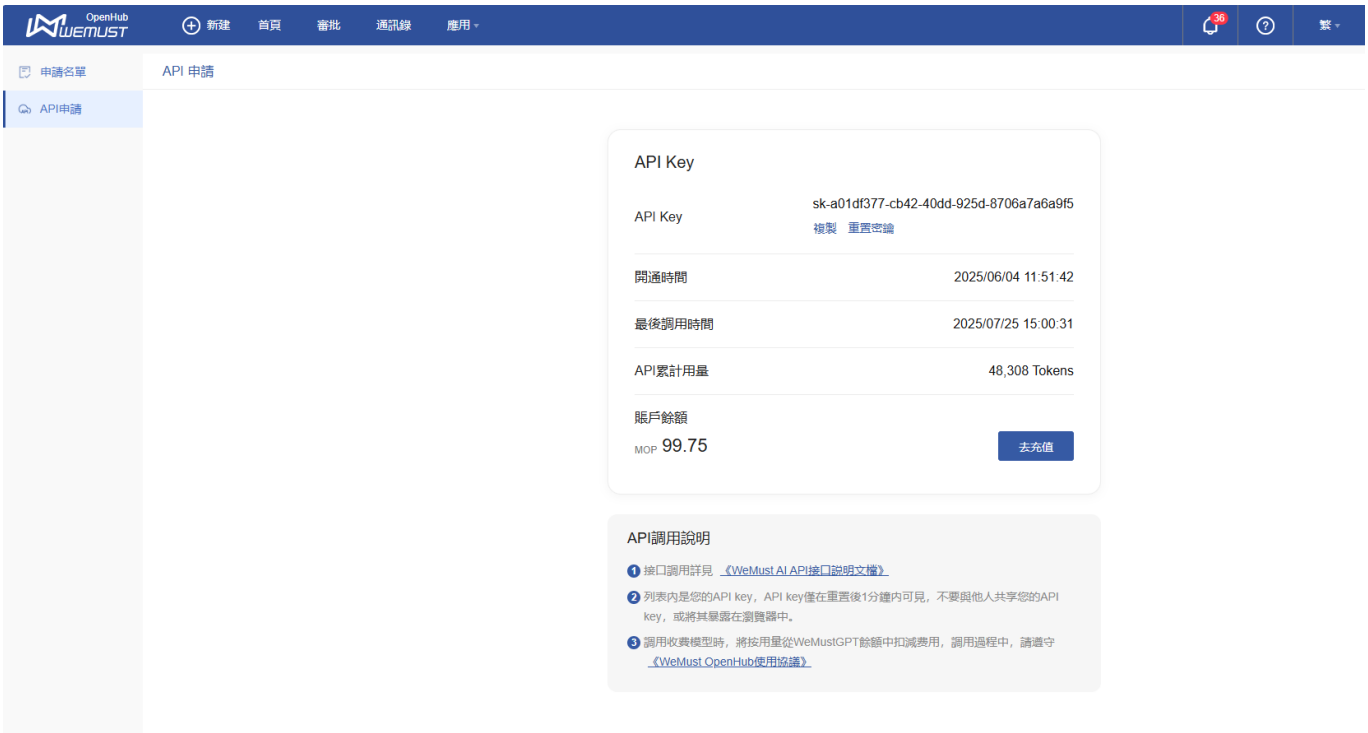


步驟2：申請API密鑰

1. 登錄後進入OpenHub

2. 點擊API申請

3. 複製生成的API密鑰並妥善保存



重要提醒：

- API密鑰就像你的密碼，不要分享給他人
- 建議將密鑰保存在安全的地方，如密碼管理器中
- 如果密鑰洩露，立即在控制台中刪除並重新生成

我是開發者：請往下看

我只想使用AI作爲我的工作助手： 跳轉1.4 我想要簡單直接的使用AI作爲各式助手：Cherry Studio

1.3 第一次API調用（5分鐘實戰）

現在讓我們進行第一次API調用！我們將創建一個簡單的Python程序，讓AI介紹一下李白這位偉大的詩人。

步驟1：安裝必要的庫

```
pip install requests
```

步驟2：創建你的第一個AI程序

```
import requests
import os

# 設置你的API密鑰（請替換為你的實際密鑰，需要加上“Bearer ”前綴）
api_key = "Bearer 你的API密鑰"

# 設置請求頭
headers = {
    "Authorization": api_key,
    "Content-Type": "application/json"
```

```
}

# 設置請求內容
data = {
    "model": "gpt-4o-mini",  # 使用性價比最高的模型
    "messages": [
        {
            "role": "system",
            "content": "你是一個聰明的百科全書小助手"
        },
        {
            "role": "user",
            "content": "請用100字左右介紹一下李白"
        }
    ],
    "temperature": 0.7,  # 控制回答的創意程度
    "max_tokens": 200    # 限制回答長度
}

# 發送請求
try:
    response = requests.post(
        "https://ai-apigateway.must.edu.mo/openhub/v1/chat/completions",
        headers=headers,
        json=data
    )

    # 檢查請求是否成功
    if response.status_code == 200:
        result = response.json()
        ai_response = result["choices"][0]["message"]["content"]
        print("AI的回答：")
        print(ai_response)

        # 顯示使用的token數量
        usage = result.get("usage", {})
        print(f"\n使用統計：")
        print(f"輸入tokens: {usage.get('prompt_tokens', 0)}")
        print(f"輸出tokens: {usage.get('completion_tokens', 0)}")
        print(f"總計tokens: {usage.get('total_tokens', 0)}")

    else:
        print(f"請求失敗，狀態碼：{response.status_code}")
        print(f"錯誤信息：{response.text}")

except Exception as e:
    print(f"發生錯誤：{e}")
```

或者使用Openai库来执行程序：

```
import os
from openai import OpenAI
```

```python
import json

client = OpenAI(
    api_key="你的API_KEY", # 无须拼接Bearer, OpenAI库会自动拼接
    base_url="https://ai-apigateway.must.edu.mo/openhub/v1"
)
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "你是一個友善的助手"},
        {"role": "user", "content": "請介紹一下澳門科技大學"}
    ],
    temperature=0.7,
    max_tokens=200
)

print("AI回覆：")
print(response.choices[0].message.content)
print(f"\n使用統計：")
print(f"輸入tokens: {response.usage.prompt_tokens}")
print(f"輸出tokens: {response.usage.completion_tokens}")
print(f"總計tokens: {response.usage.total_tokens}")
```

步驟3：運行程序

1. 將上面的代碼保存為`my_first_ai.py`
2. 將代碼中的"你的API密鑰"替換為你實際的API密鑰，加上Bearer 前綴
3. 在終端中運行：`python my_first_ai.py`

如果一切順利，你應該會看到AI對李白的介紹，以及使用的token統計信息。恭喜你！你已經成功完成了第一次AI API調用。 使用Https 请求与OpenAI库均能够调用OpenHub API，为了使用的普适性，以下示例均用HTTPS请求来完成。

1.4 我想要簡單直接的使用AI作爲各式助手：Cherry Studio

步驟1：下載Cherry Studio 前往官網：https://www.cherry-ai.com/download

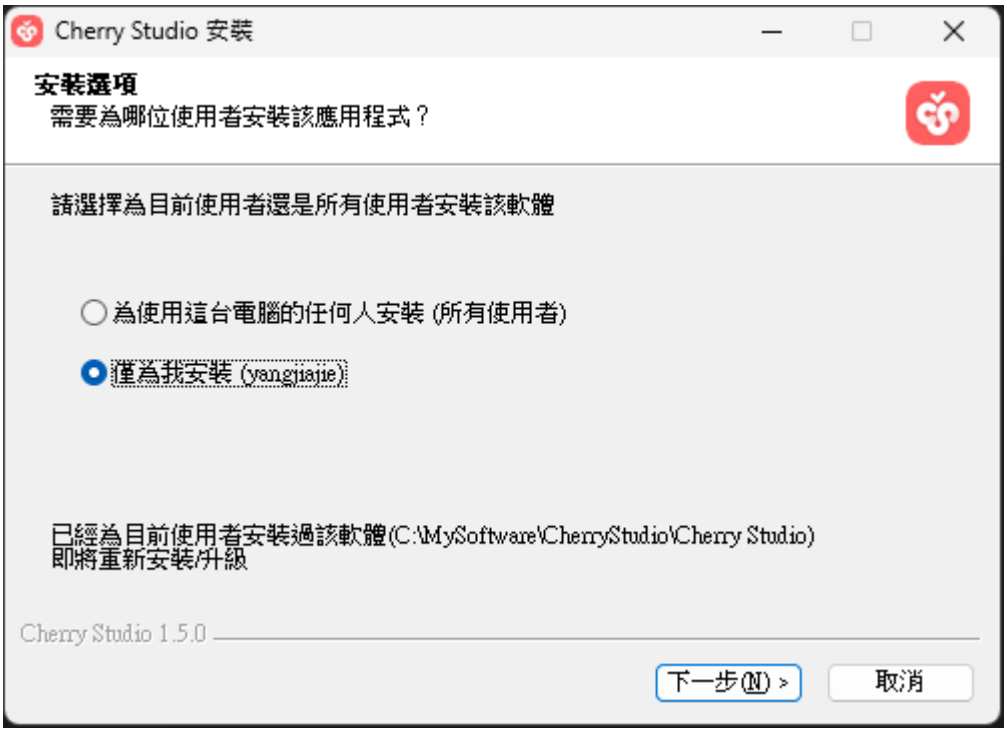
官網外觀如下：

見到此頁面往下：

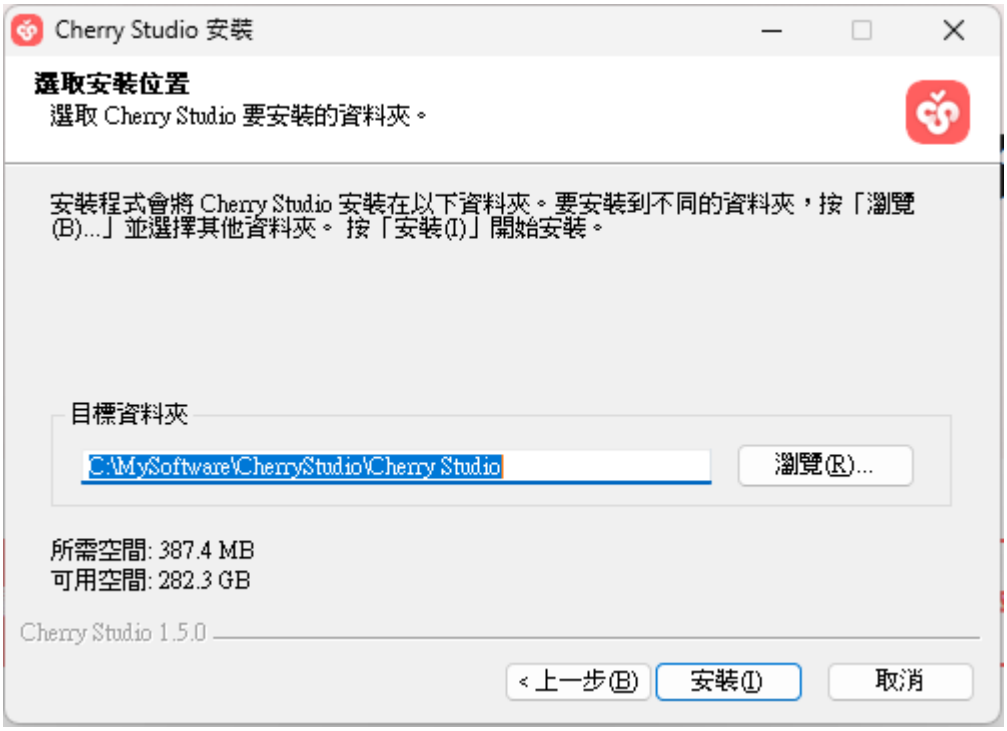
下載對應系統的安裝包，一般來説，Windows只需要下載標準版即可。蘋果電腦需要根據自身處理器的芯片決定下載哪一個版本。若其中一個無法運行，下載另一個即可。

步驟2：安裝Cherry Studio

打開下載好的Cherry Studio 安裝包



點擊下一步

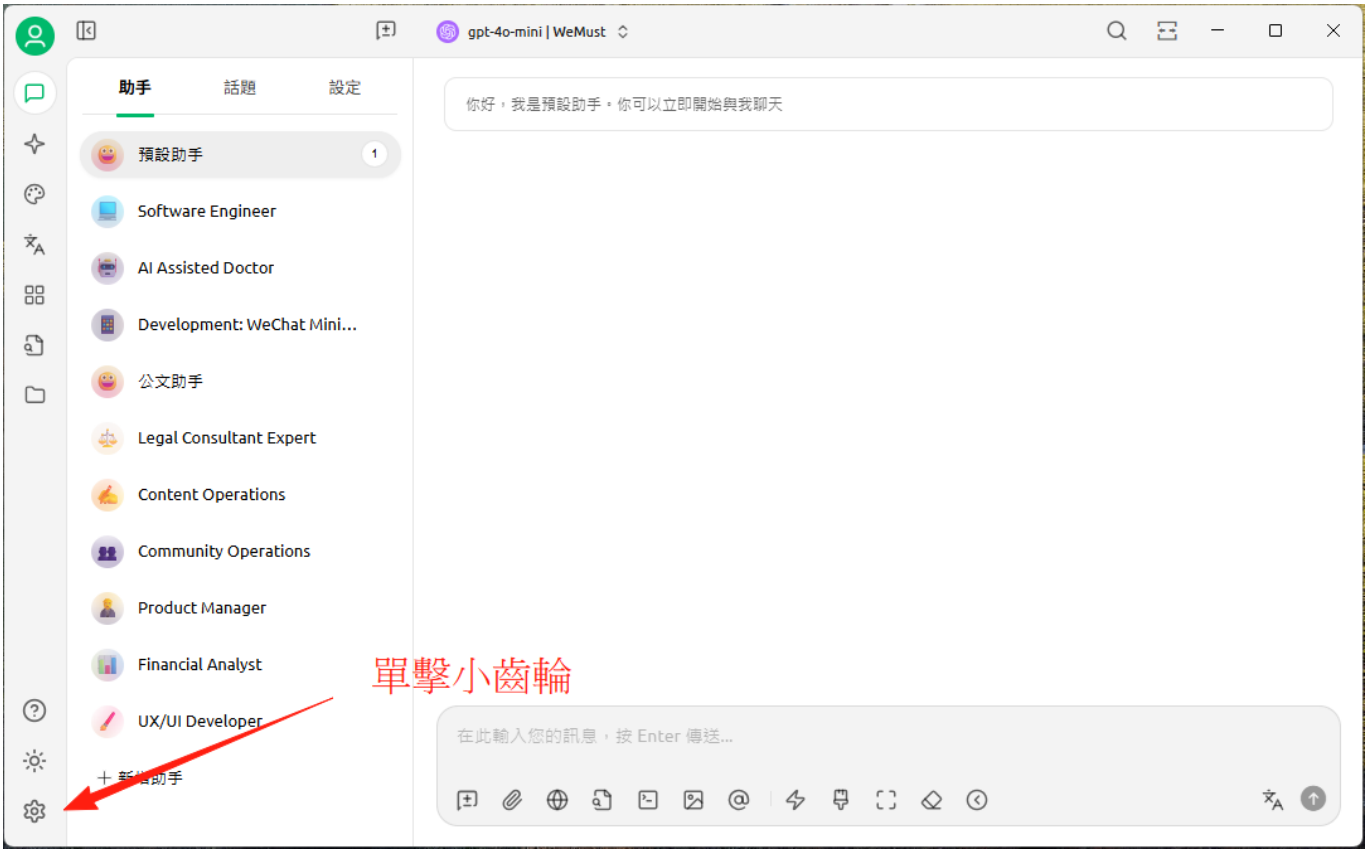


點擊安裝。

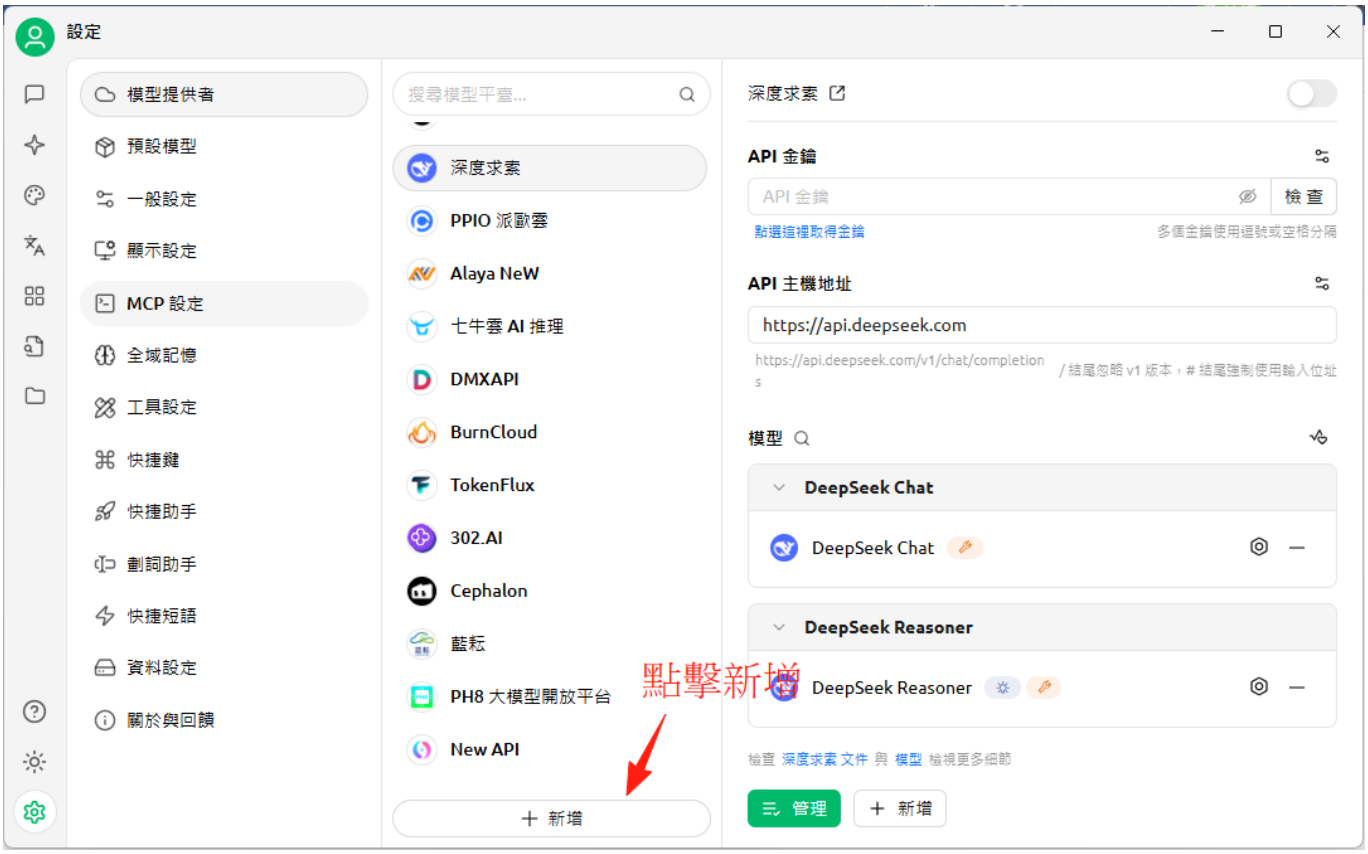
（可選）更改安裝路徑。但由於程序體積不大在默認安裝路徑也沒問題

安裝完成後，桌面會出現Cherry Studio 的快捷方式。雙擊快捷方式

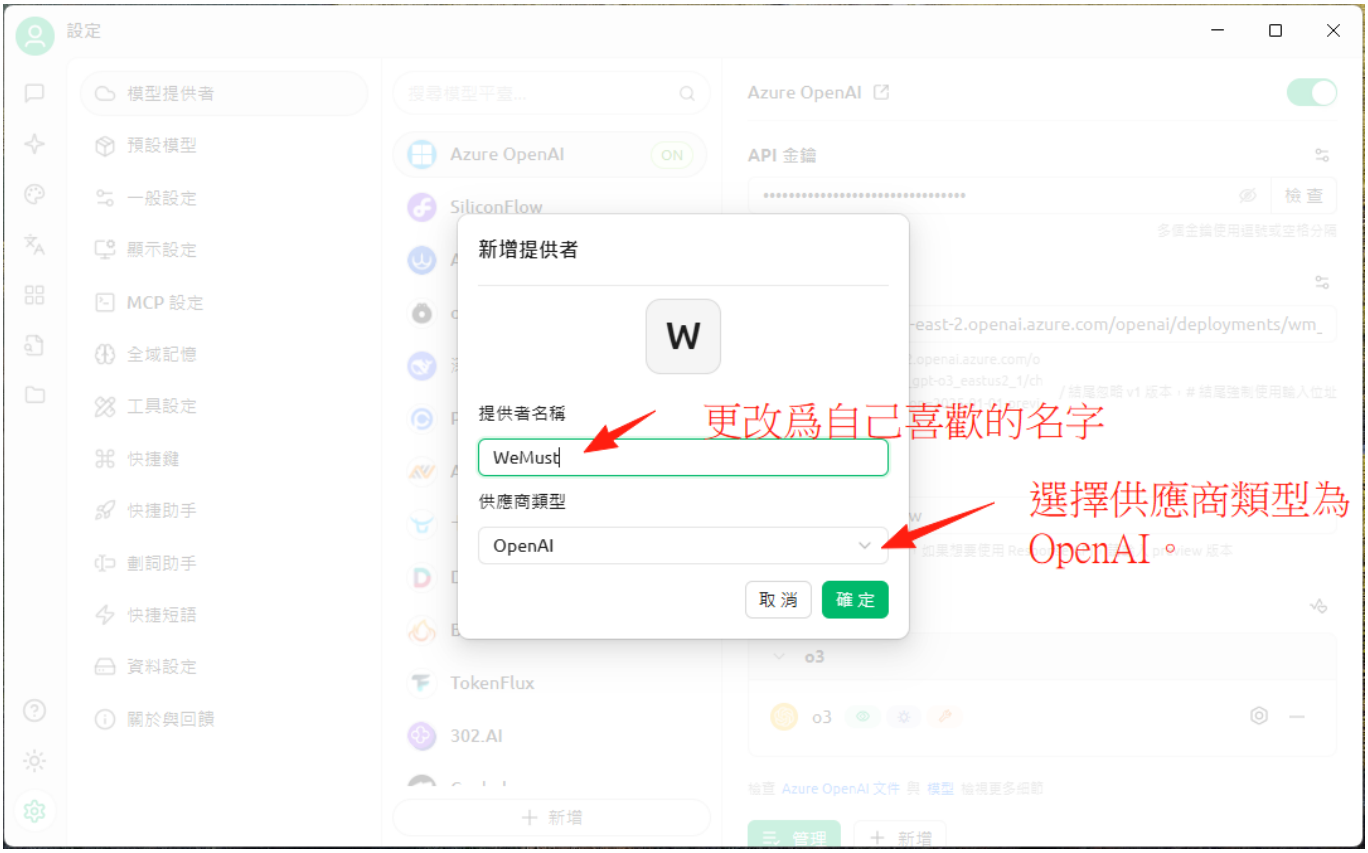
步驟3：配置Cherry Studio



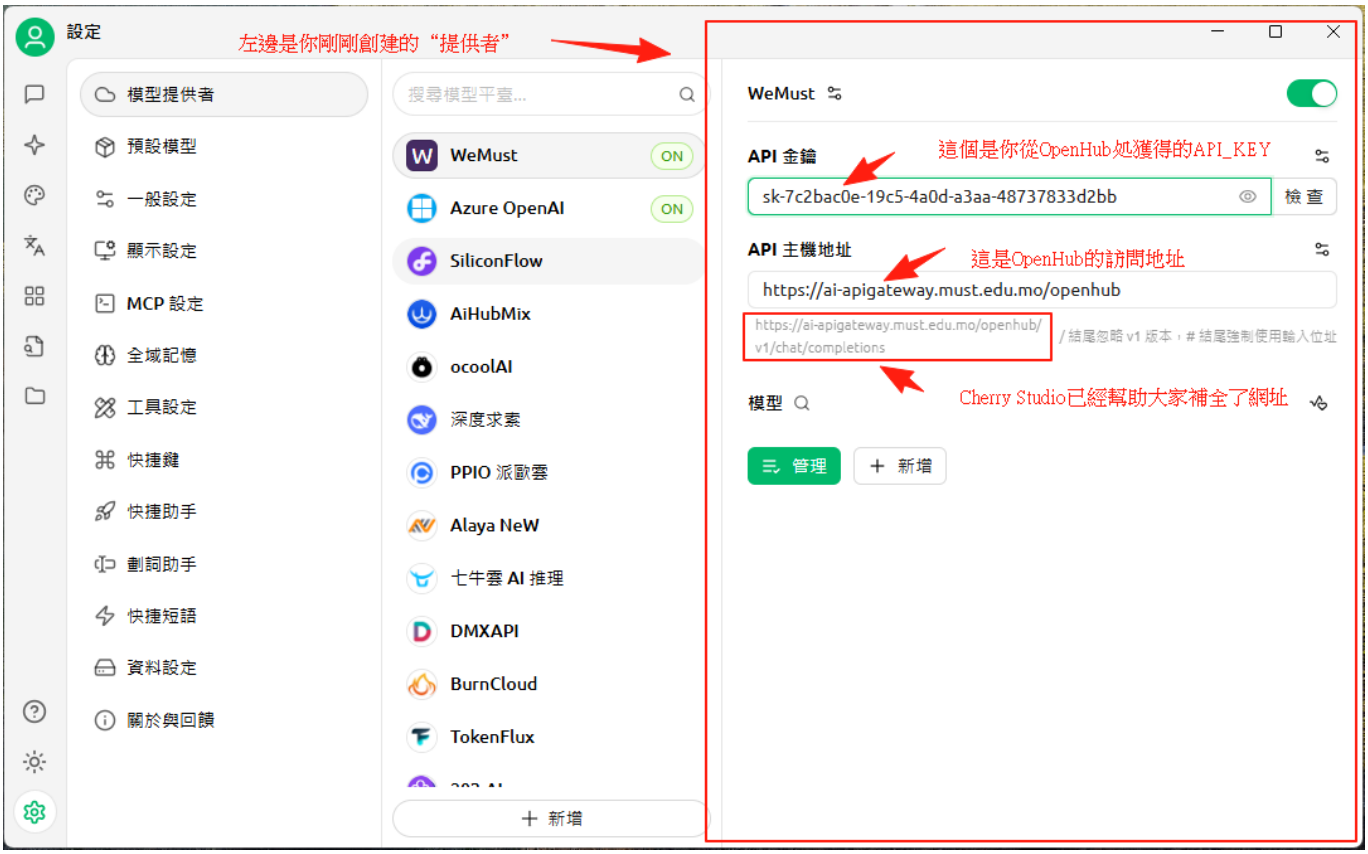
進入Cherry Studio 程序後，點擊左下角小齒輪，如圖中所示，進入以下界面，點擊“+新增”



出現如下界面：



供應商類型選擇為OpenAI,並更改提供者名稱，點擊確定后，出現以下界面：

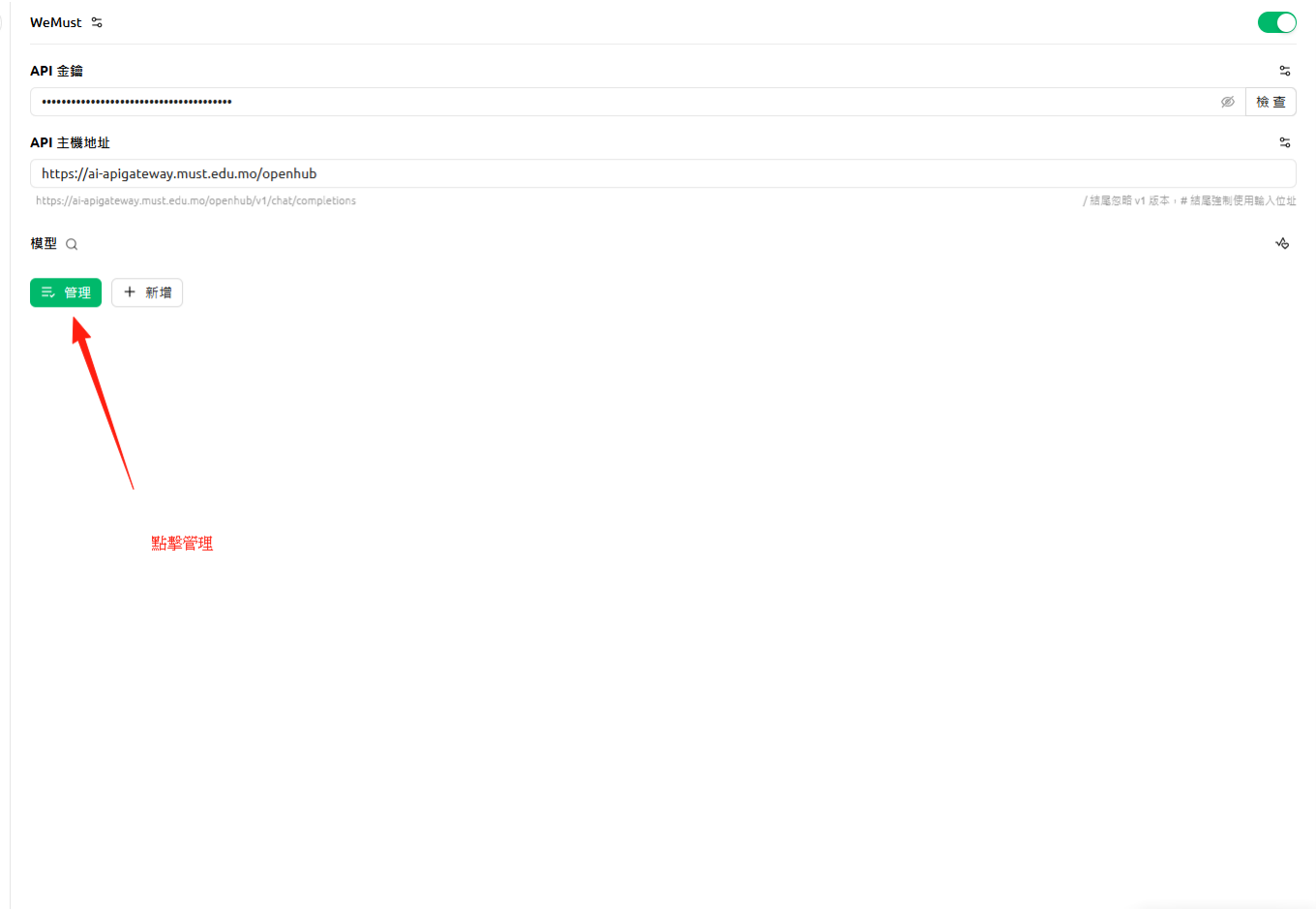


API 金鑰中輸入你從OpenHub中獲得的API KEY，並在API主機地址中複製粘貼：

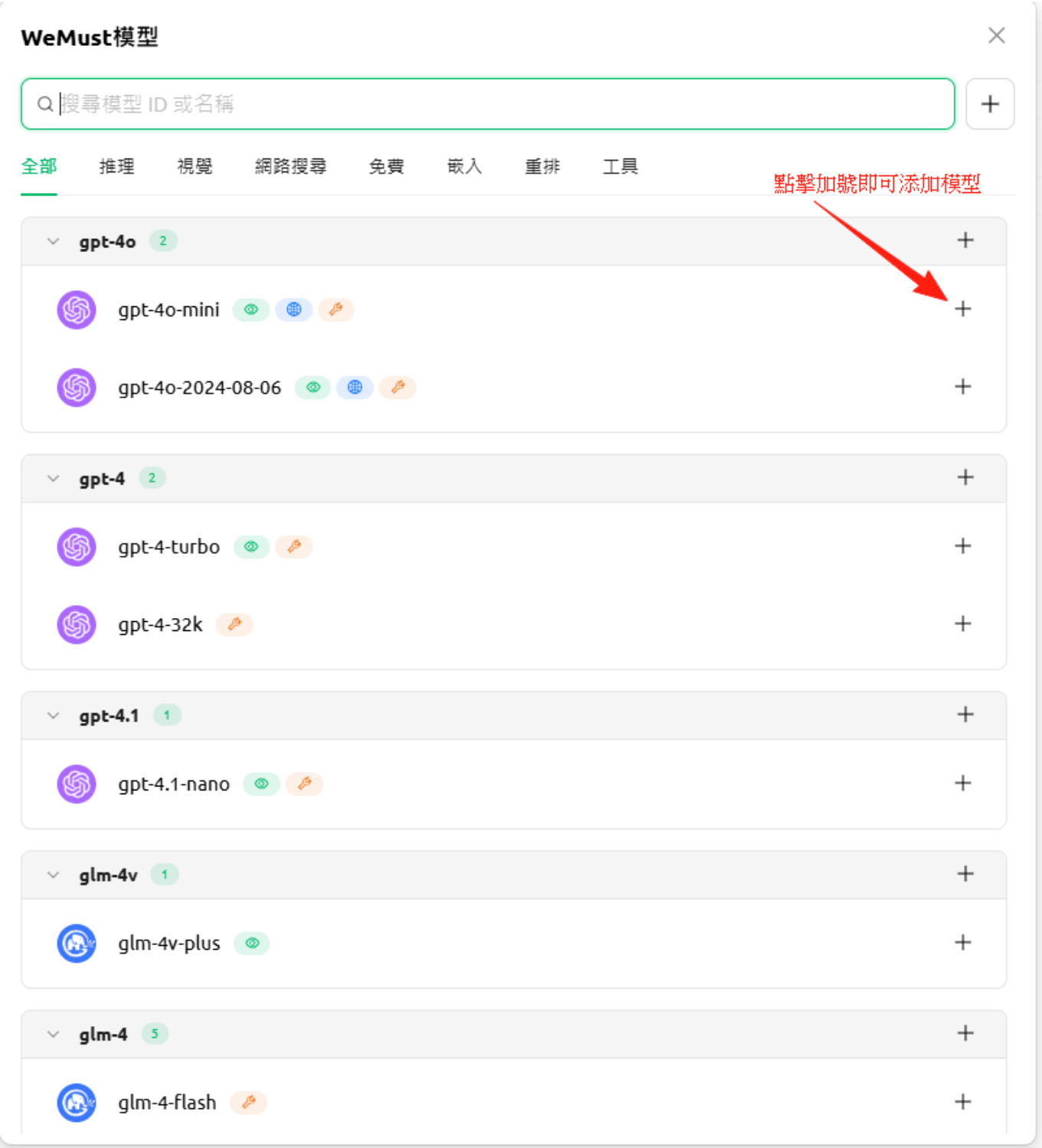
https://ai-apigateway.must.edu.mo/openhub

💡 注意圖中的灰色地址，這是Cherry Studio自動補全的地址，應該顯示：https://ai-apigateway.must.edu.mo/openhub/v1/chat/completions

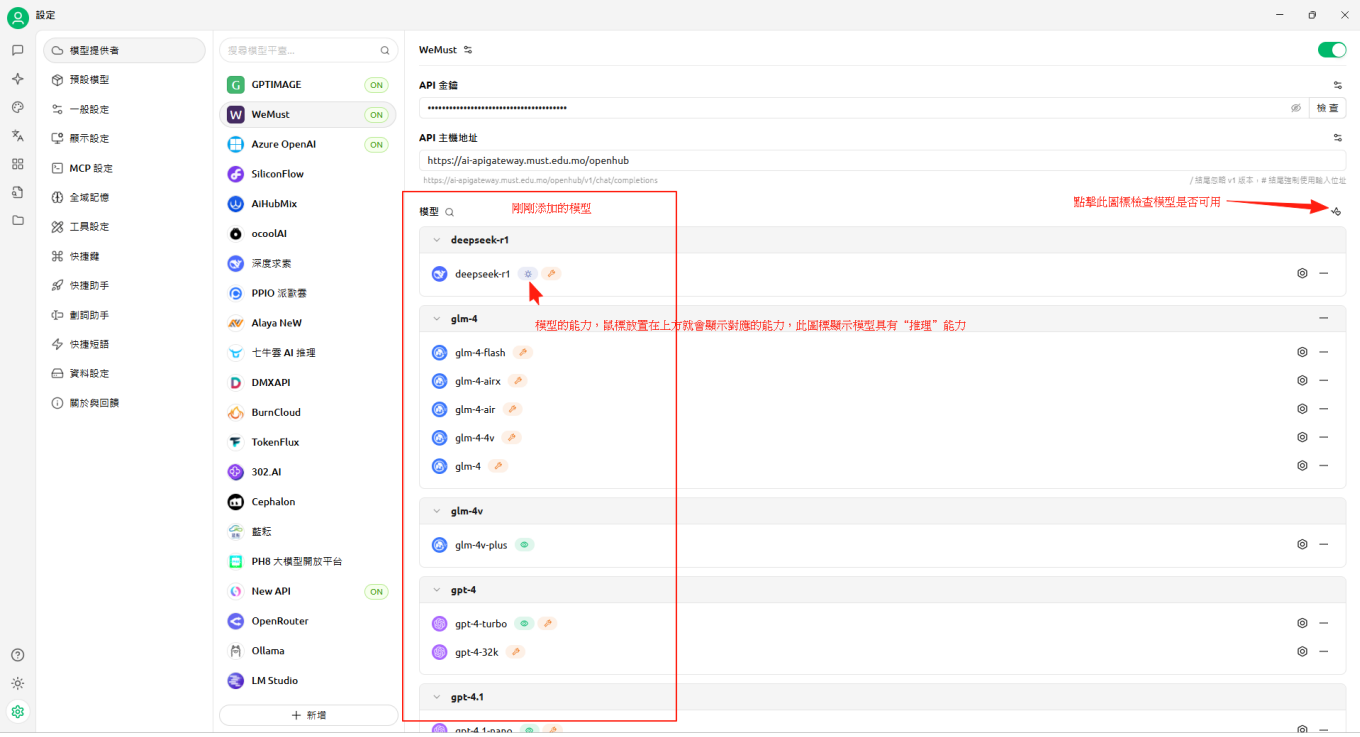
輸入完成后，點擊管理


WeMust
API 金鑰
檢查
API 主機地址
https://ai-apigateway.must.edu.mo/openhub
https://ai-apigateway.must.edu.mo/openhub/v1/chat/completions
/ 結尾忽略 v1 版本，# 結尾強制使用輸入位址
模型
管理
新增
點擊管理

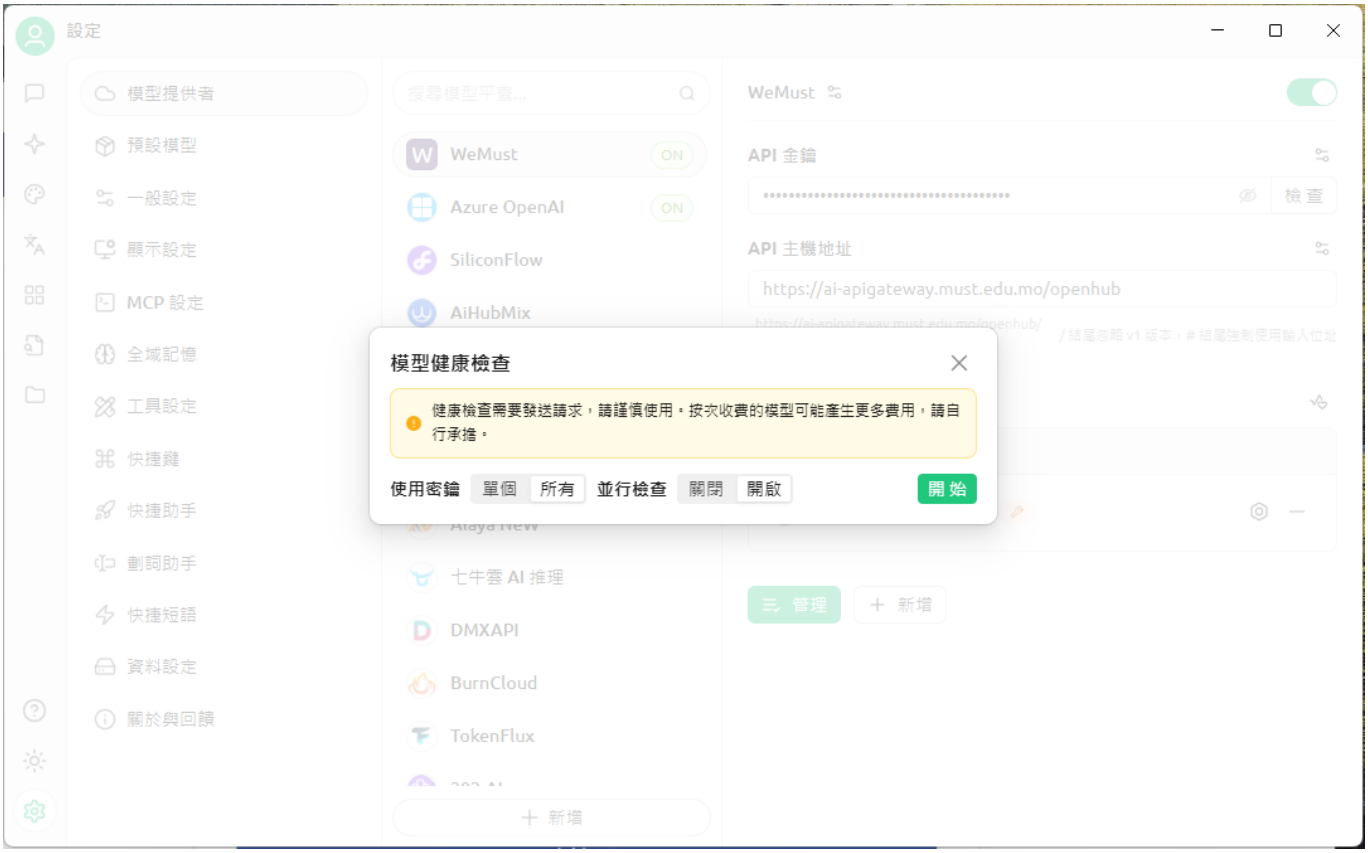
若你的API主機地址和API金鑰均正確，管理頁面會出現當前可用的所有模型，如下所示：



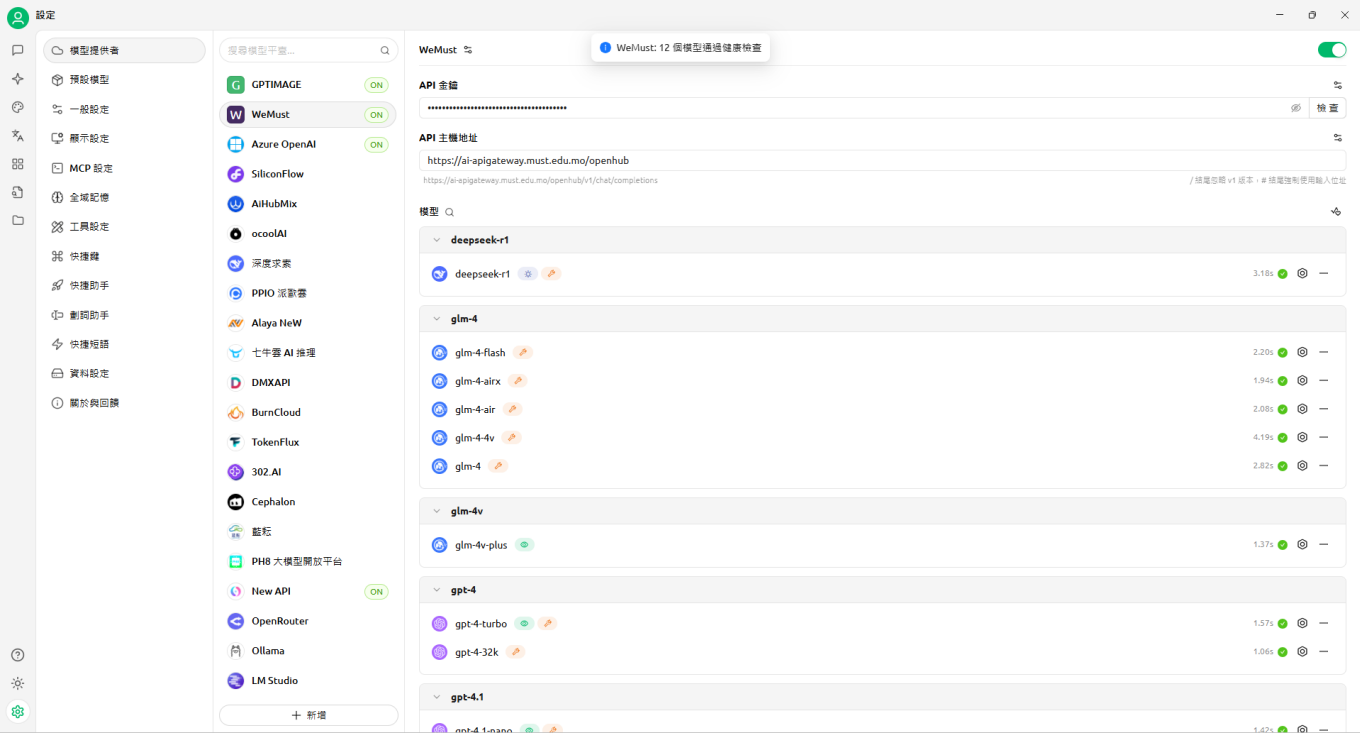
如果該頁面為空白，請檢查API KEY和API主機地址是否正確填寫。點擊+號，添加相應模型，各個模型的能力參考及推薦：6.4 模型能力對比表



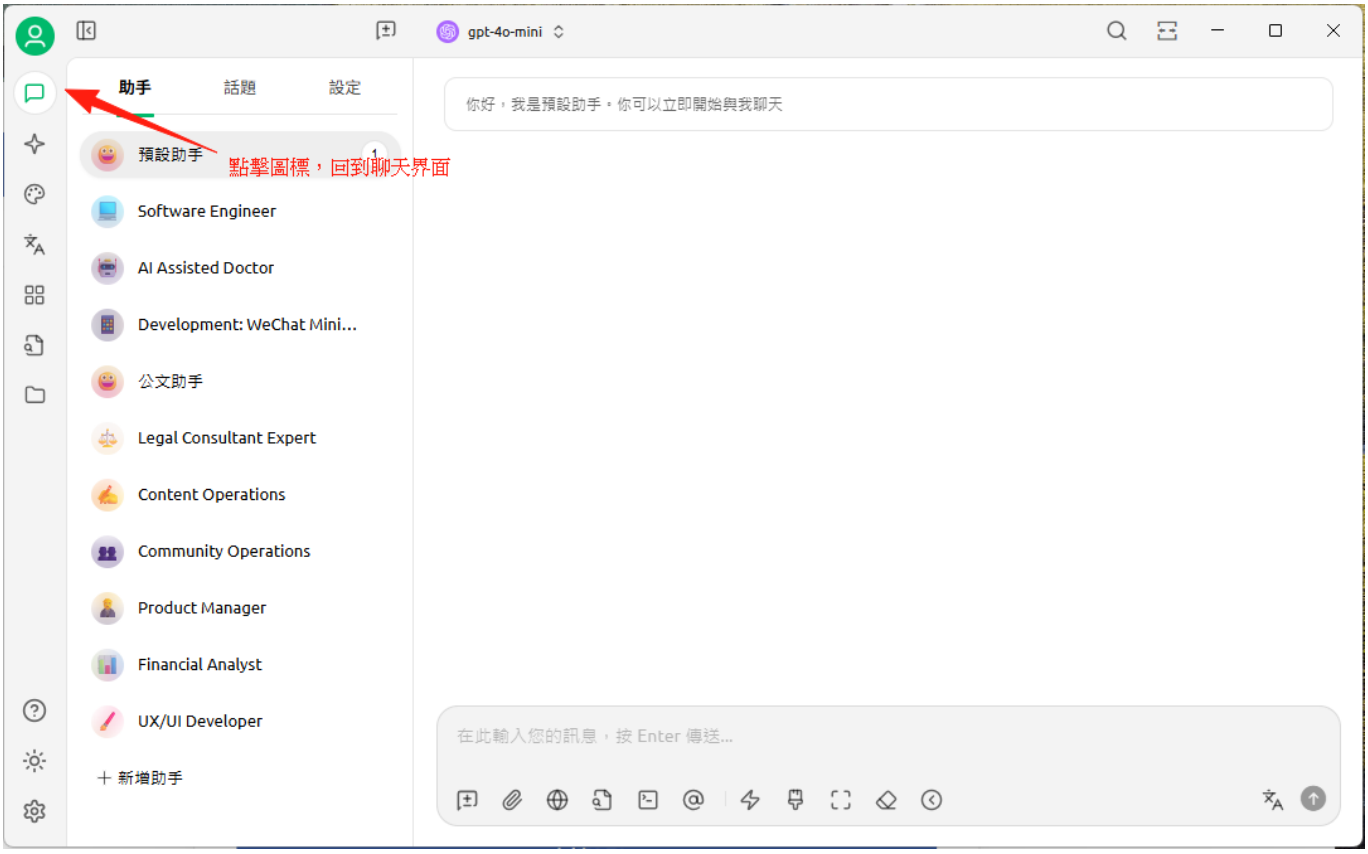
點擊圖上所示的按鈕進行測試，出現如下界面：

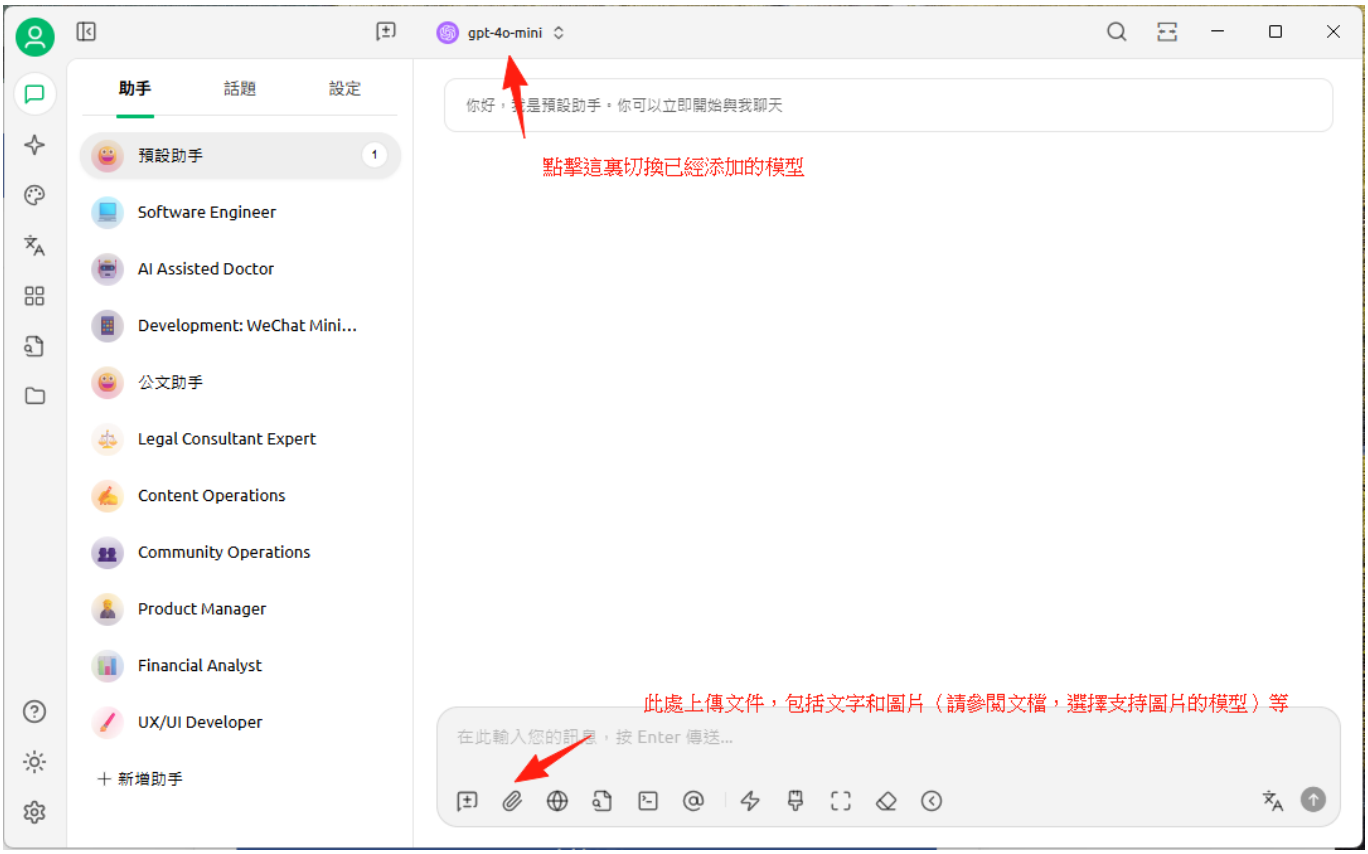


點擊開始

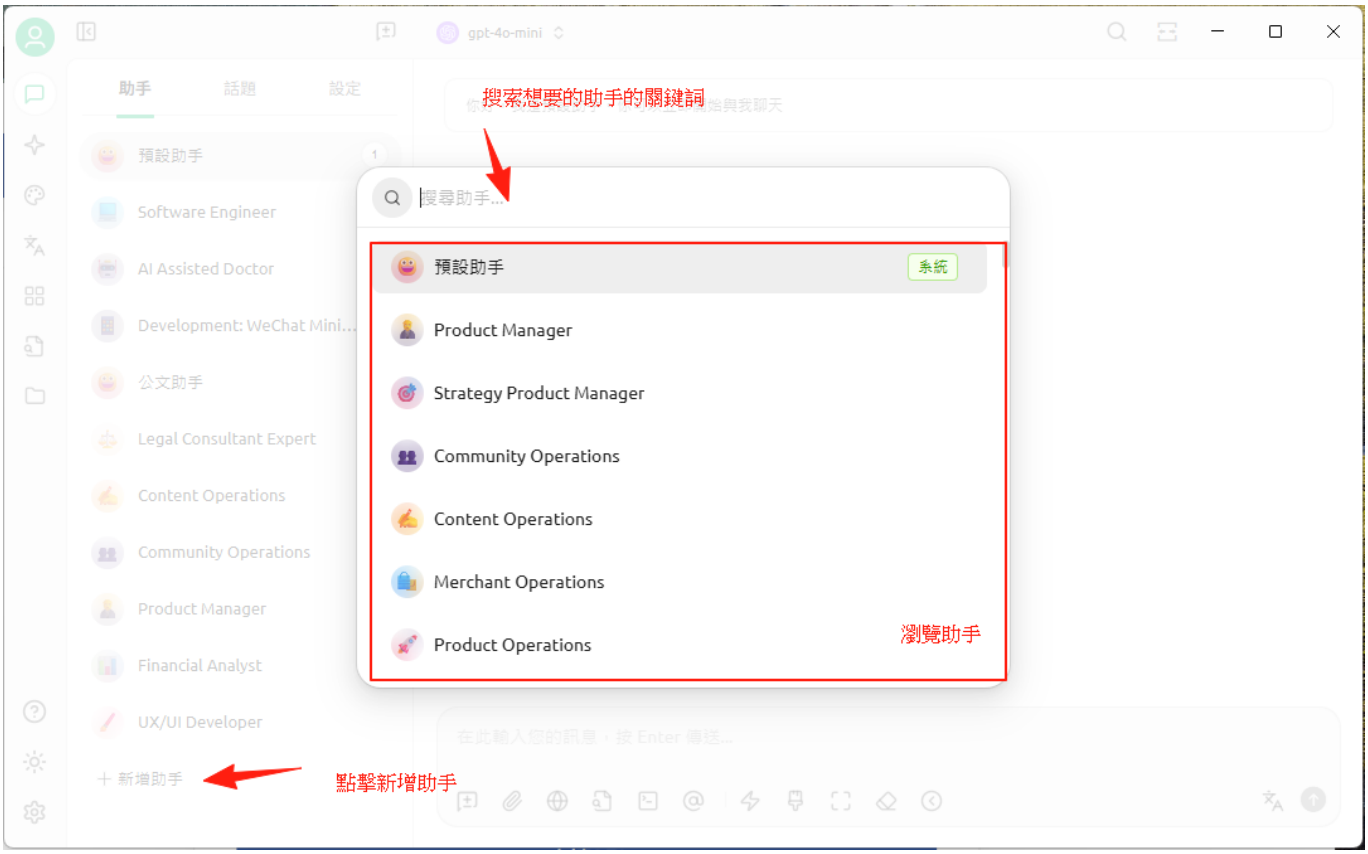


出現上述界面，表示配置成功，回到聊天界面：

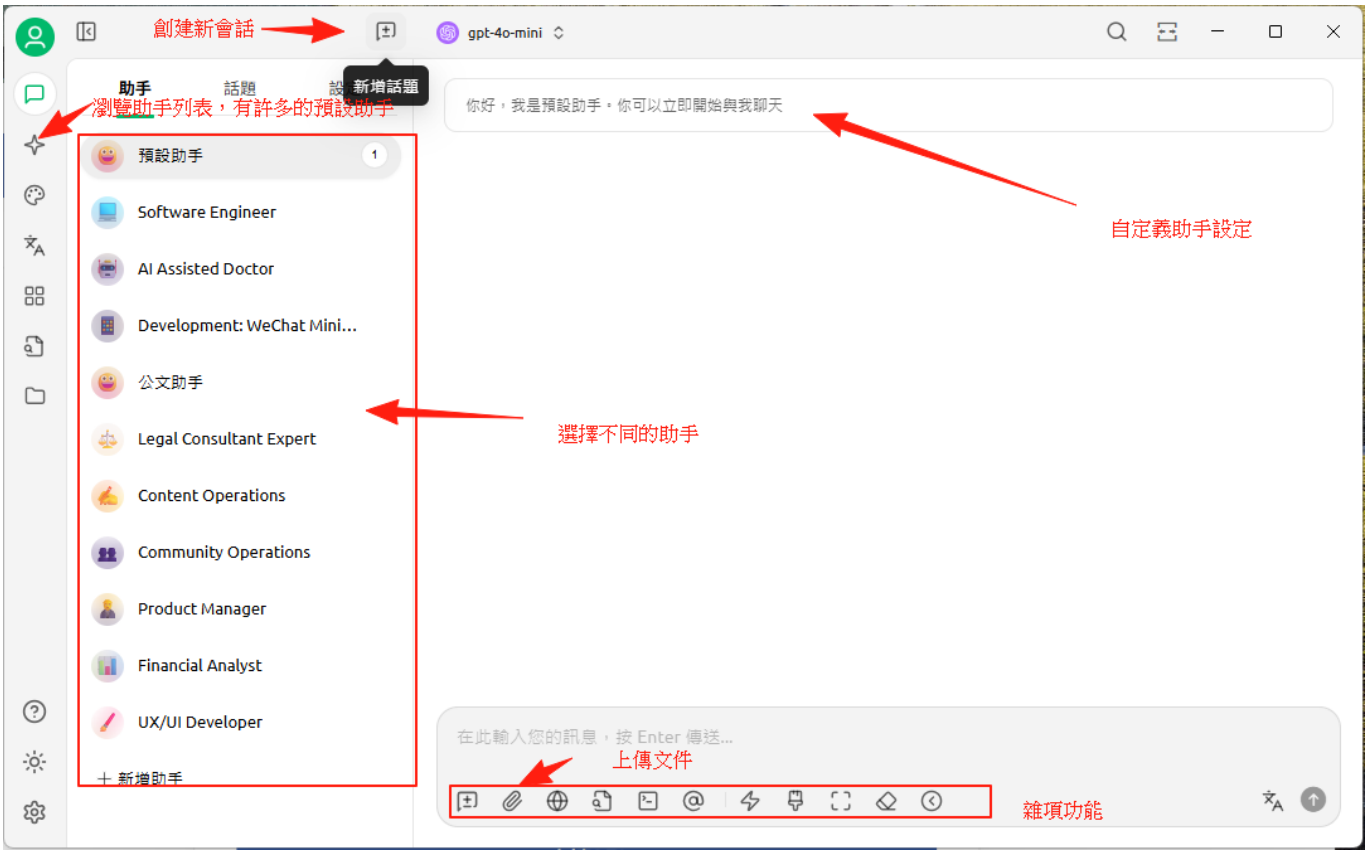


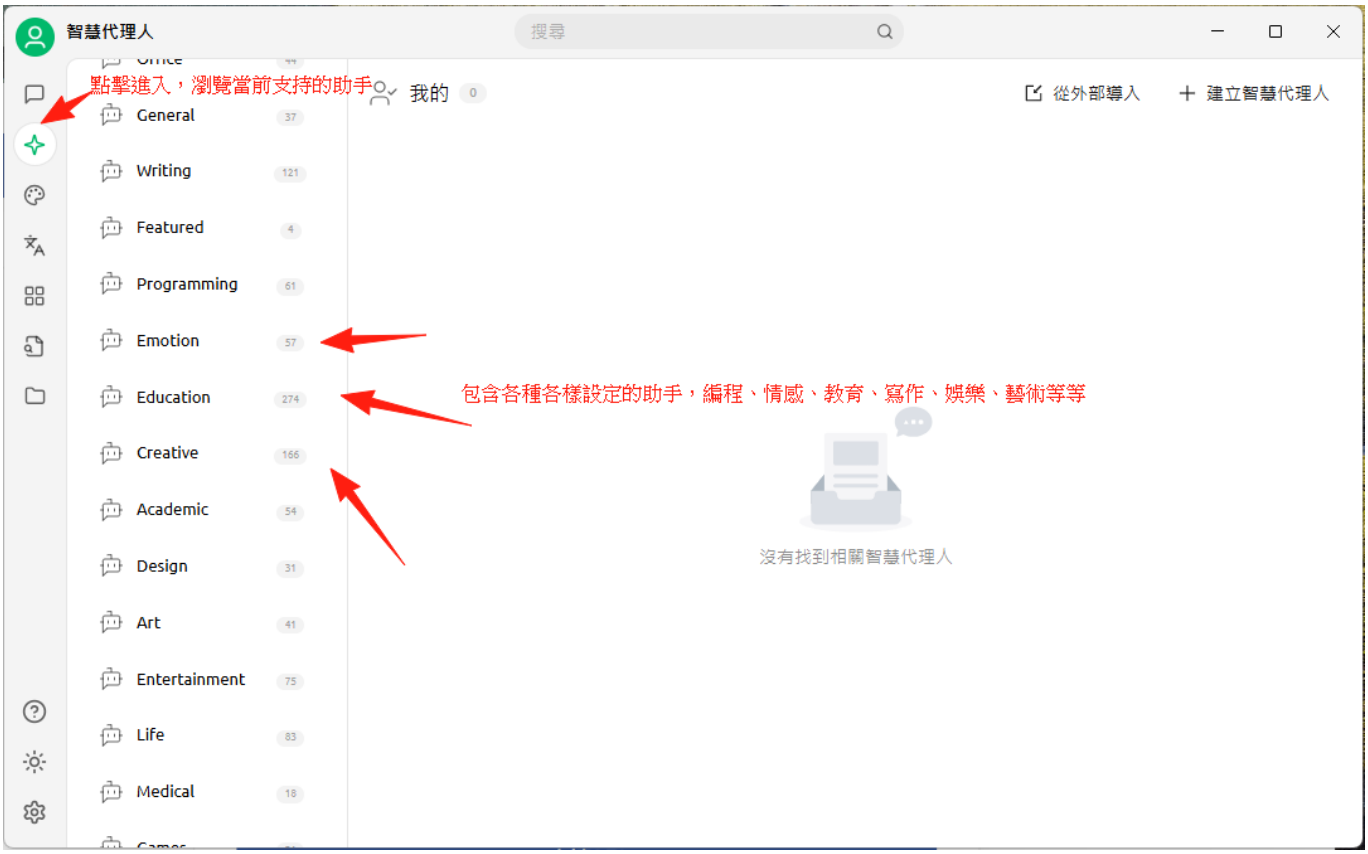


你可以選擇不同的助手：

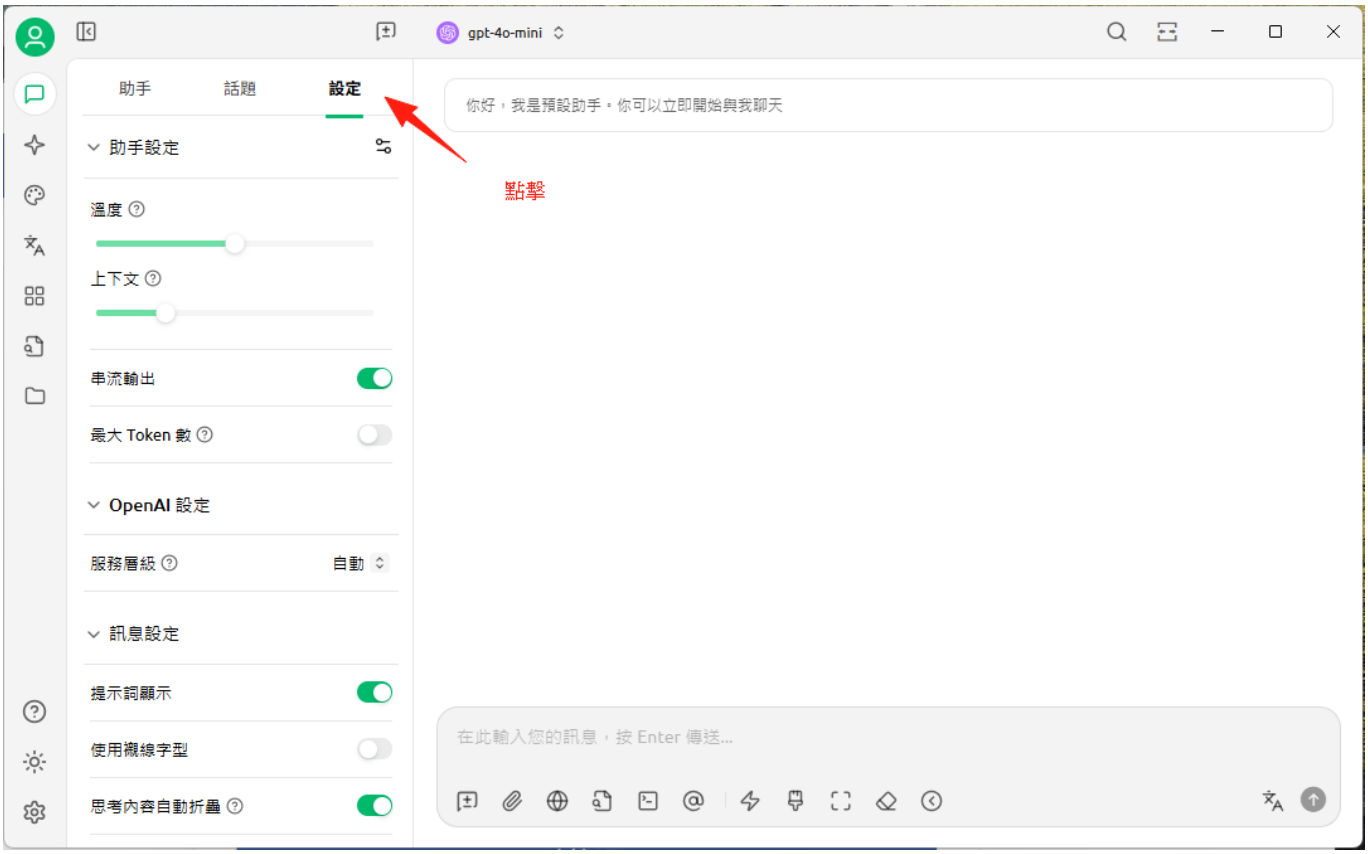


你可以瀏覽目前的cherry studio中，有哪些預設的助手：





如果你已經讀過本教程的第四章：第四章：參數調優指南 。你可以嘗試調整參數已適應自己的需求。當然，默認的參數能夠適應大部分的場景：

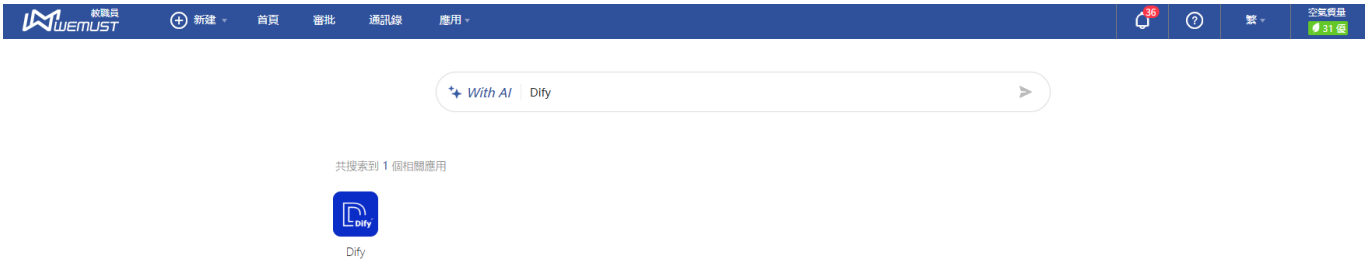


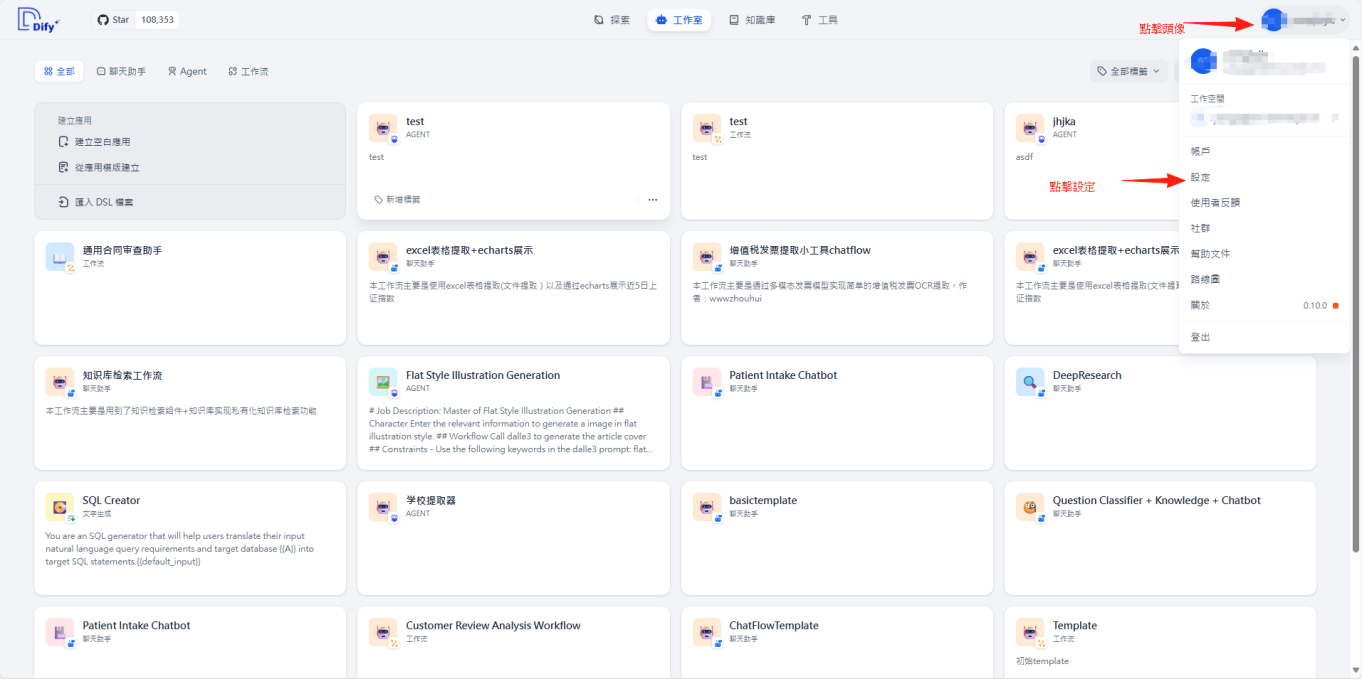
現在，你已經可以通過Cherry studio使用OpenHub API了！

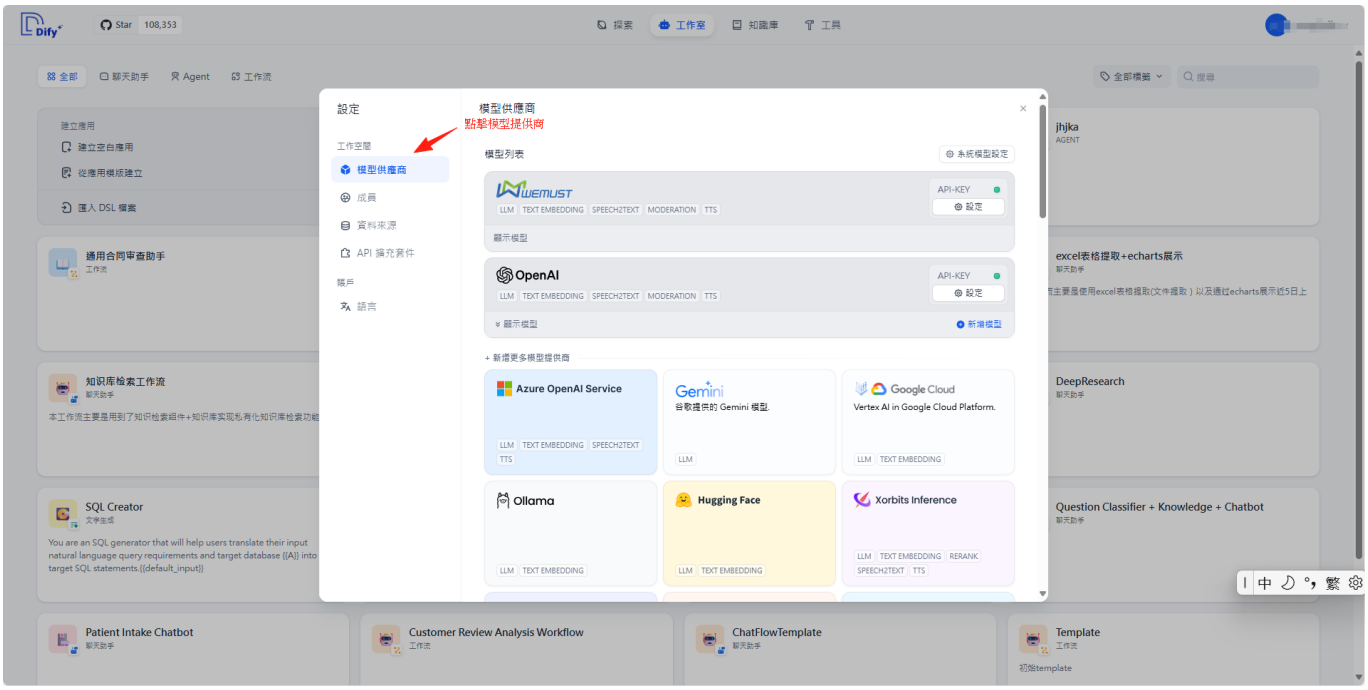
Cherry Studio中一些有趣的Agent:

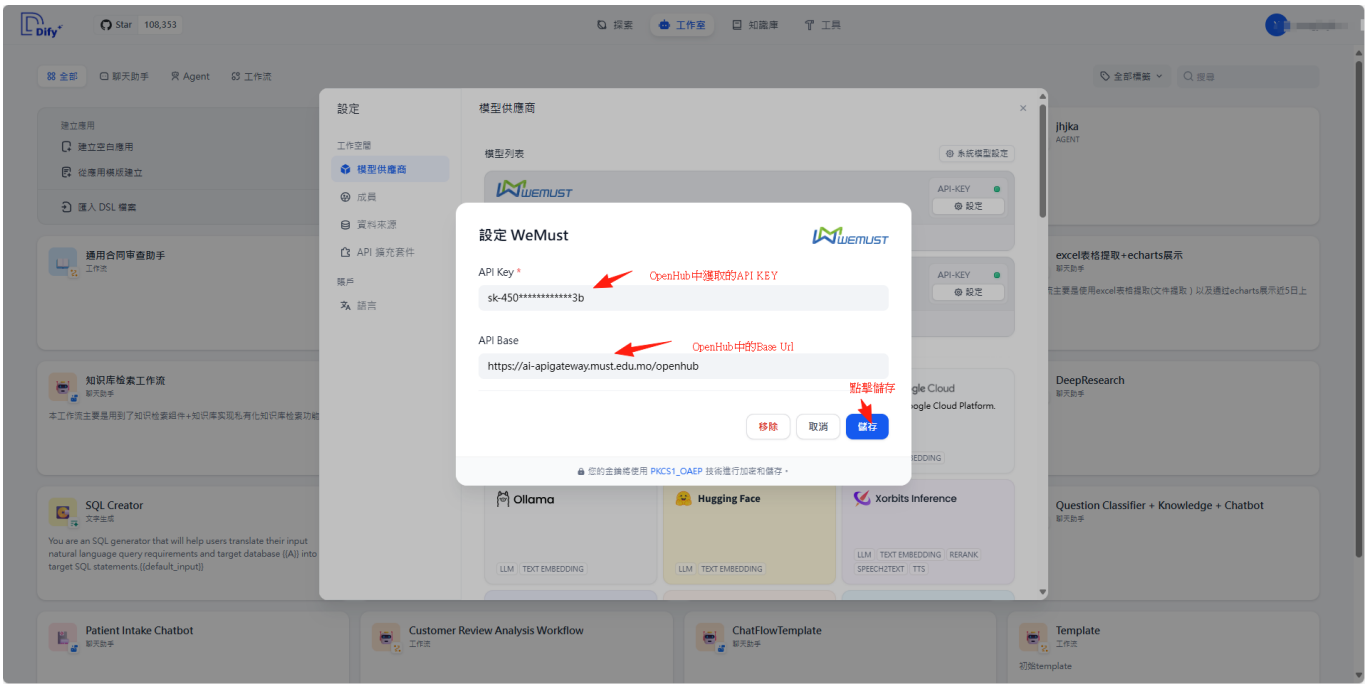
1.6 Dify配置

在WeMust中進入Dify：

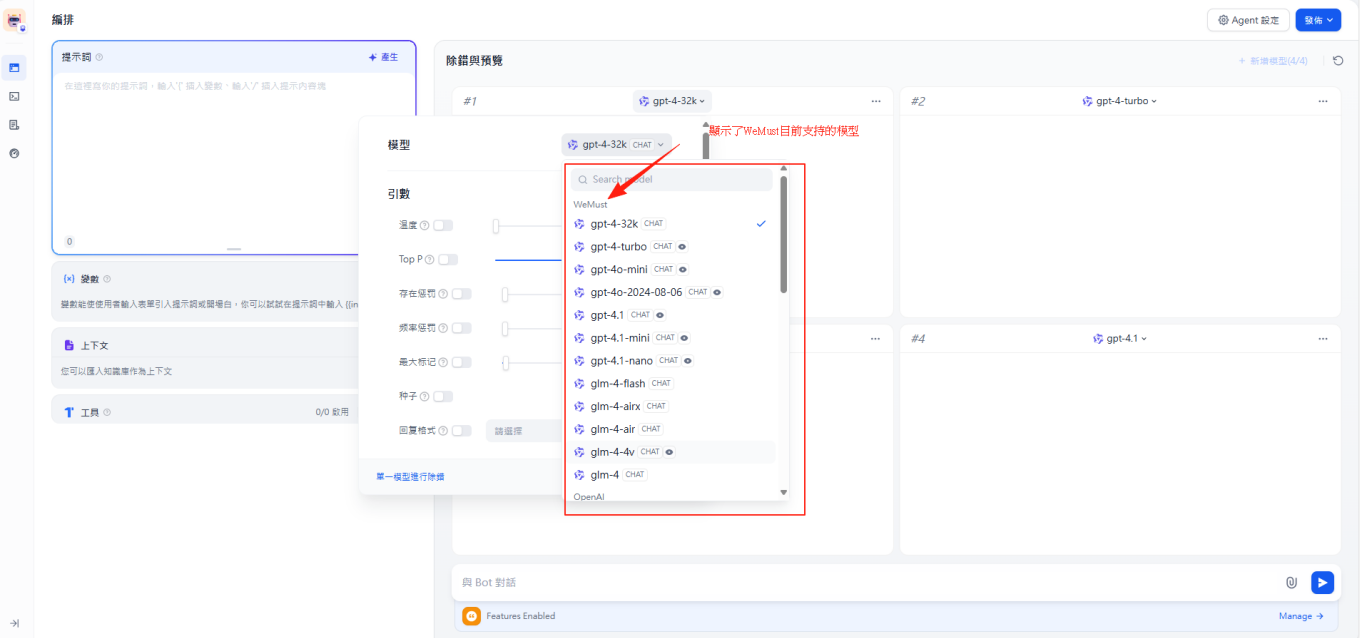


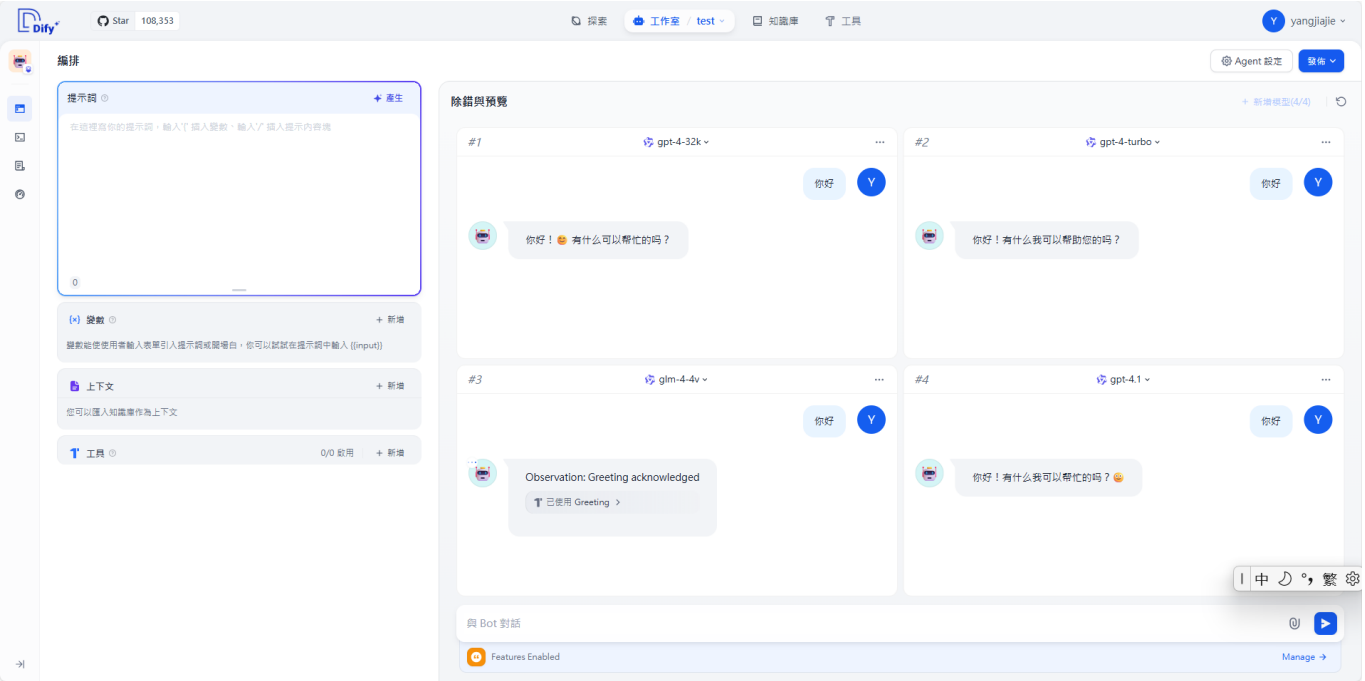

點擊頭像
點擊設定
工作室
知識庫
工具
探索
全部
聊天助手
Agent
工作流
建立應用
建立空白應用
從應用模版建立
匯入 DSL 檔案
test
jhjka
通用合同審查助手
excel表格提取+echarts展示
增值稅發票提取小工具chatflow
知識庫檢索工作流
Flat Style Illustration Generation
Patient Intake Chatbot
DeepResearch
SQL Creator
学校提取器
basictemplate
Question Classifier + Knowledge + Chatbot
Customer Review Analysis Workflow
ChatFlowTemplate
Template
帳戶
設定
使用者反饋
社群
幫助文件
路線圖
關於
登出


點擊模型提供商
設定
工作空間
模型供應商
成員
資料來源
API 擴充資料
帳戶
語言
模型列表
系統模型設定
WEMUST
OpenAI
API-KEY
設定
顯示模型
新增模型
+ 新增更多模型提供商
Azure OpenAI Service
Gemini
谷歌提供的 Gemini 模型.
Google Cloud
Vertex AI in Google Cloud Platform.
Ollama
Hugging Face
Xorbits Inference


設定 WeMust
API Key *
sk-450**********3b
OpenHub中獲取的API KEY
API Base
https://ai-apigateway.must.edu.mo/openhub
OpenHub中的Base Url
點擊儲存
移除
取消
儲存
模型供應商
模型列表
Ollama
Hugging Face
Xorbits Inference

測試：





1.5 常見問題快速解決

問題1：收到401錯誤（認證失敗）

- 檢查API密鑰是否正確複製
- 確認密鑰前後沒有多餘的空格
- 驗證密鑰是否已過期或被禁用

問題2：收到400錯誤（請求參數錯誤）

- 檢查模型名稱是否正確（注意大小寫）
- 確認JSON格式是否正確
- 驗證必需參數是否都已提供

問題3：請求超時

- 檢查網絡連接是否正常

- 嘗試增加請求超時時間
- 確認API服務是否正常運行

問題4：中文顯示亂碼

- 確保Python文件使用UTF-8編碼保存
- 在文件開頭添加：`# -*- coding: utf-8 -*-`
- 檢查終端是否支持中文顯示

快速測試命令 如果你想快速測試API是否可用，可以使用curl命令：

```
curl -X POST "https://ai-apigateway.must.edu.mo/openhub/v1/chat/completions" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer 你的API密鑰" \
  -d '{
    "model": "gpt-4o-mini",
    "messages": [{"role": "user", "content": "你好"}],
    "max_tokens": 50
  }'
```

通過這個快速開始指南，你已經掌握了WeMust AI API的基本使用方法。接下來的章節將深入介紹API的各種功能和高級用法，幫助你構建更複雜和實用的AI應用。

第二章：API基礎知識

2.1 API概述和架構

API（Application Programming Interface，應用程序編程接口）是不同軟件組件之間通信的橋樑。WeMust AI API採用RESTful架構設計，這意味著它使用標準的HTTP方法（如GET、POST）來處理請求，並以JSON格式交換數據。

API架構圖：

```
你的應用程序 → HTTP請求 → WeMust AI API → AI模型處理 → 返回結果 → 你的應用程序
```

核心概念：

1. **端點（Endpoint）**：API的具體地址，每個端點負責特定的功能
2. **請求（Request）**：你發送給API的數據和指令
3. **響應（Response）**：API返回給你的結果
4. **認證（Authentication）**：證明你有權使用API的機制
5. **模型（Model）**：執行AI任務的具體算法，如GPT-4o、GLM-4等

API版本管理： WeMust AI API目前使用v1版本，所有API路徑都以`/v1`開頭。這種版本管理方式確保了向後兼容性，即使API更新，你的現有代碼仍然可以正常工作。

2.2 認證機制詳解

WeMust AI API提供兩種認證方式，適用於不同的使用場景。理解這兩種方式的區別和適用場景，對於選擇合適的認證方法非常重要。

2.2.1 API密鑰認證

API密鑰認證是最簡單直接的認證方式，特別適合學習和開發階段使用。你只需要在每次請求的HTTP頭部包含你的API密鑰即可。

使用方法：

```
headers = {
    "Authorization": "Bearer 你的API密鑰",
    "Content-Type": "application/json"
}
```

注意事項：

- 需要添加"Bearer "前綴
- 密鑰區分大小寫，請確保完全正確
- 每個密鑰都有使用配額限制

適用場景：

- 個人學習和開發
- 小型項目和原型開發
- 服務器端應用（密鑰不會暴露給用戶）

2.3 基礎URL和端點

WeMust AI API提供了兩個環境供不同階段使用：

環境對比表：

環境	基礎URL	用途
PROD（生產環境）	`https://ai-apigateway.must.edu.mo/openhub/`	正式使用

主要端點：

1. **聊天補全端點**
    - URL: `/v1/chat/completions`
    - 方法: POST
    - 功能: 創建AI對話和內容生成

完整的API調用URL示例：

```
https://ai-apigateway.must.edu.mo/openhub/v1/chat/completions
```

2.4 支持的AI模型介紹

WeMust AI API支持多種先進的AI模型，每個模型都有其特點和適用場景。選擇合適的模型對於獲得最佳效果和控制成本都很重要。

模型對比表：

模型名稱	特點	適用場景	性價比	推薦指數
gpt-4o-mini	快速、經濟	日常對話、簡單任務	★★★★★	★★★★★
gpt-4o-2024-08-06	平衡 性能	通用任務 、內容創作	★★★★	★★★★★
gpt-4-turbo	高性能	複雜推理、專業任務	★★★	★★★★
gpt-4-32k	長文本處理	文檔分析、長對話	★★	★★★
glm-4	中文優化	中文內容處理	★★★★	★★★★
glm-4v-plus	多模態	圖文理解、視覺任務	★★★	★★★★
deepseek-r1	推理專用	數學、邏輯推理	★★★	★★★

- 完整的模型对比表见 6.4 模型能力對比表

模型選擇建議：

1. **初學者推薦**：`gpt-4o-mini`
   - 成本最低，響應速度快
   - 適合學習和實驗
   - 能滿足大部分基本需求
2. **專業應用**：`gpt-4-turbo`
   - 最強的推理能力
   - 適合複雜的專業任務
   - 成本較高，需要謹慎使用
3. **中文優化**：`glm-4`系列
   - 對中文理解更準確
   - 適合中文內容創作
   - 支持中國文化背景

模型使用示例：

```
# 不同場景的模型選擇
scenarios = {
    "日常聊天": {
        "model": "gpt-4o-mini",
        "temperature": 0.7,
        "max_tokens": 500
    },
```

```
    "創意寫作": {
        "model": "gpt-4-turbo",
        "temperature": 0.8,
        "max_tokens": 2000
    },
    "中文詩詞": {
        "model": "glm-4",
        "temperature": 0.6,
        "max_tokens": 800
    }
}
```

通過理解這些基礎知識，你已經具備了使用WeMust AI API的理論基礎。接下來的章節將深入介紹具體的功能使用方法，幫助你構建實際的AI應用。

第三章：核心功能使用

3.1 聊天對話功能

聊天對話是WeMust AI API的核心功能，它允許你創建智能對話系統。理解對話的基本結構和角色設定，是構建高質量AI應用的關鍵。

3.1.1 基本對話實現

每個對話都由一系列消息組成，每條消息都有特定的角色。理解這些角色的作用，能幫助你更好地控制AI的行為。

消息角色說明：

1. **system（系統角色）**：設定AI的行為和性格
2. **user（用戶角色）**：用戶的輸入和問題
3. **assistant（助手角色）**：AI的回覆和響應

基本對話結構：

```
messages = [
    {
        "role": "system",
        "content": "你是一個專業的程序設計助手，擅長解釋複雜的技術概念"
    },
    {
        "role": "user",
        "content": "請解釋什麼是API"
    }
]
```

完整的對話實現：

```python
import requests
import json

def chat_with_ai(messages, model="gpt-4o-mini"):
    """
    與AI進行對話的基本函數

    Args:
        messages: 對話消息列表
        model: 使用的AI模型

    Returns:
        AI的回覆內容
    """
    api_key = "Bearer 你的API密鑰"

    headers = {
        "Authorization": api_key,
        "Content-Type": "application/json"
    }

    data = {
        "model": model,
        "messages": messages,
        "temperature": 0.7,
        "max_tokens": 1000
    }

    try:
        response = requests.post(
            "https://ai-apigateway.must.edu.mo/openhub/v1/chat/completions",
            headers=headers,
            json=data
        )

        if response.status_code == 200:
            result = response.json()
            return result["choices"][0]["message"]["content"]
        else:
            return f"錯誤：{response.status_code} - {response.text}"

    except Exception as e:
        return f"請求失敗：{e}"

# 使用示例
messages = [
    {
        "role": "system",
        "content": "你是一個友善的學習助手，用簡單易懂的語言回答問題"
    },
    {
        "role": "user",
        "content": "什麼是機器學習？"
```

```
    }
]

response = chat_with_ai(messages)
print("AI回答：", response)
```

3.1.2 系統角色設定

系統角色（system role）是控制AI行為的關鍵工具。通過精心設計的系統提示，你可以讓AI扮演不同的角色，具備特定的知識和風格。

系統角色設計原則：

1. **明確性**：清楚地描述AI應該如何行為
2. **具體性**：提供具體的指導和限制
3. **一致性**：確保角色設定在整個對話中保持一致

常用系統角色模板：

```
# 學習助手
system_tutor = """
你是一個專業的學習助手，具有以下特點：
1. 用簡單易懂的語言解釋複雜概念
2. 提供實際例子幫助理解
3. 鼓勵學生思考和提問
4. 如果不確定答案，會誠實地說明
5. 總是保持耐心和友善的態度
"""

# 代碼助手
system_coder = """
你是一個經驗豐富的程序員，專門幫助解決編程問題：
1. 提供清晰、可運行的代碼示例
2. 解釋代碼的工作原理
3. 指出潛在的問題和改進建議
4. 遵循最佳編程實踐
5. 支持多種編程語言
"""

# 創意寫作助手
system_writer = """
你是一個富有創意的寫作助手：
1. 能夠創作各種類型的文本內容
2. 注重文字的美感和表達力
3. 根據需求調整寫作風格
4. 提供建設性的寫作建議
5. 保持原創性和創新性
"""

# 客服助手
system_support = """
```

```
你是一個專業的客服代表：
1. 始終保持禮貌和專業
2. 耐心聆聽客戶問題
3. 提供準確和有用的信息
4. 如果無法解決問題，會引導客戶尋求進一步幫助
5. 記住客戶的需求和偏好
"""
```

動態角色切換示例：

```
class AIAssistant:
    def __init__(self, api_key):
        self.api_key = api_key
        self.conversation_history = []

    def set_role(self, role_type):
        """設置AI角色"""
        roles = {
            "tutor": "你是一個專業的學習助手...",
            "coder": "你是一個經驗豐富的程序員...",
            "writer": "你是一個富有創意的寫作助手...",
            "support": "你是一個專業的客服代表..."
        }

        system_message = {
            "role": "system",
            "content": roles.get(role_type, roles["tutor"])
        }

        # 重置對話歷史，添加新的系統角色
        self.conversation_history = [system_message]

    def chat(self, user_input):
        """進行對話"""
        # 添加用戶輸入
        self.conversation_history.append({
            "role": "user",
            "content": user_input
        })

        # 調用API
        response = self.call_api()

        # 添加AI回覆到歷史
        if response:
            self.conversation_history.append({
                "role": "assistant",
                "content": response
            })

        return response
```

```
    def call_api(self):
        """調用API的具體實現"""
        # 這裡是API調用代碼...
        pass

# 使用示例
assistant = AIAssistant("你的API密鑰")

# 設置為學習助手
assistant.set_role("tutor")
response1 = assistant.chat("解釋一下什麼是遞歸")

# 切換為代碼助手
assistant.set_role("coder")
response2 = assistant.chat("寫一個計算斐波那契數列的Python函數")
```

3.1.3 對話歷史管理

在多輪對話中，管理對話歷史是非常重要的。AI需要記住之前的對話內容才能提供連貫的回覆。在WeMust GPT中，我們已經幫你完成了這一步驟，但在你自己的開發項目中，你必須自己管理歷史對話

對話歷史的重要性：

- 保持對話的連貫性
- 讓AI理解上下文
- 支持複雜的多輪交互
- 提供個性化的體驗

對話歷史管理策略：

```
class ConversationManager:
    def __init__(self, max_history=10):
        self.messages = []
        self.max_history = max_history

    def add_system_message(self, content):
        """添加系統消息"""
        self.messages = [{"role": "system", "content": content}]

    def add_user_message(self, content):
        """添加用戶消息"""
        self.messages.append({"role": "user", "content": content})
        self._trim_history()

    def add_assistant_message(self, content):
        """添加助手消息"""
        self.messages.append({"role": "assistant", "content": content})
        self._trim_history()

    def _trim_history(self):
        """修剪對話歷史，保持在限制範圍內"""
```

```
        # 保留系統消息
        system_messages = [msg for msg in self.messages if msg["role"] ==
"system"]
        other_messages = [msg for msg in self.messages if msg["role"] != "system"]

        # 如果超出限制，移除最舊的非系統消息
        if len(other_messages) > self.max_history:
            other_messages = other_messages[-self.max_history:]

        self.messages = system_messages + other_messages

    def get_messages(self):
        """獲取當前對話歷史"""
        return self.messages

    def clear_history(self, keep_system=True):
        """清除對話歷史"""
        if keep_system:
            system_messages = [msg for msg in self.messages if msg["role"] ==
"system"]
            self.messages = system_messages
        else:
            self.messages = []

# 使用示例
conversation = ConversationManager(max_history=6)

# 設置系統角色
conversation.add_system_message("你是一個數學老師，擅長解釋數學概念")

# 模擬多輪對話
conversation.add_user_message("什麼是質數？")
conversation.add_assistant_message("質數是只能被1和自己整除的大於1的自然數...")

conversation.add_user_message("請舉幾個質數的例子")
conversation.add_assistant_message("前幾個質數是：2, 3, 5, 7, 11, 13...")

conversation.add_user_message("如何判斷一個數是否為質數？")
# AI會記住之前關於質數的討論

print("當前對話歷史：")
for msg in conversation.get_messages():
    print(f"{msg['role']}: {msg['content'][:50]}...")
```

3.2 流式響應處理

流式響應讓你的應用能夠實時顯示AI的回覆，就像ChatGPT那樣逐字顯示內容。這大大改善了用戶體驗，特別是在處理長回覆時。

3.2.1 什麼是流式響應

傳統的API調用需要等待AI完成整個回覆後才返回結果，而流式響應則是AI一邊生成內容一邊發送給你，讓用戶能夠立即看到回覆的開始部分。

流式響應的優勢：

- 更好的用戶體驗（即時反饋）
- 減少感知延遲
- 支持長文本生成
- 可以提前終止不需要的回覆

流式響應的工作原理：

```
傳統方式：請求 → 等待 → 完整回覆
流式方式：請求 → 部分回覆1 → 部分回覆2 → ... → 完成
```

3.2.2 實現實時對話效果

基本流式響應實現：

```python
import requests
import json

def stream_chat(messages, model="gpt-4o-mini"):
    """
    實現流式對話

    Args:
        messages: 對話消息列表
        model: 使用的AI模型

    Yields:
        每個生成的文本片段
    """
    api_key = "Bearer 你的API密鑰"

    headers = {
        "Authorization": api_key,
        "Content-Type": "application/json"
    }

    data = {
        "model": model,
        "messages": messages,
        "stream": True,  # 啟用流式響應
        "temperature": 0.7,
        "max_tokens": 1000
    }

    try:
        response = requests.post(
```

```
            "https://ai-apigateway.must.edu.mo/openhub/v1/chat/completions",
            headers=headers,
            json=data,
            stream=True  # 重要：啟用流式接收
        )

        if response.status_code == 200:
            for line in response.iter_lines():
                if line:
                    # 解碼並處理每一行
                    line = line.decode('utf-8').strip()

                    # 檢查是否是數據行
                    if line.startswith('data:'):
                        data_content = line.split('data:', 1)[1].strip()

                        # 檢查是否是結束標記
                        if data_content == '[DONE]':
                            break

                        try:
                            # 解析JSON數據
                            chunk = json.loads(data_content)

                            # 提取內容
                            if 'choices' in chunk and len(chunk['choices']) > 0:
                                delta = chunk['choices'][0].get('delta', {})
                                if 'content' in delta:
                                    yield delta['content']

                        except json.JSONDecodeError:
                            continue

        else:
            yield f"錯誤：{response.status_code} - {response.text}"

    except Exception as e:
        yield f"請求失敗：{e}"

# 使用示例
messages = [
    {
        "role": "system",
        "content": "你是一個故事講述者，擅長創作有趣的故事"
    },
    {
        "role": "user",
        "content": "請講一個關於勇敢小貓的故事"
    }
]

print("AI正在創作故事：")
full_response = ""
```

```
for chunk in stream_chat(messages):
    print(chunk, end='', flush=True)  # 實時顯示
    full_response += chunk

print("\n\n故事完成！")
```

進階流式響應處理：

```
import time
import threading
from queue import Queue

class StreamingChatBot:
    def __init__(self, api_key):
        self.api_key = api_key
        self.is_streaming = False

    def stream_with_callback(self, messages, on_chunk=None, on_complete=None):
        """
        帶回調函數的流式響應

        Args:
            messages: 對話消息
            on_chunk: 每個文本片段的回調函數
            on_complete: 完成時的回調函數
        """
        def stream_worker():
            full_response = ""

            try:
                for chunk in self._stream_request(messages):
                    if not self.is_streaming:
                        break

                    full_response += chunk

                    if on_chunk:
                        on_chunk(chunk, full_response)

                if on_complete:
                    on_complete(full_response)

            except Exception as e:
                if on_complete:
                    on_complete(f"錯誤：{e}")

            finally:
                self.is_streaming = False

        self.is_streaming = True
        thread = threading.Thread(target=stream_worker)
        thread.start()
```

```
        return thread

    def stop_streaming(self):
        """停止流式響應"""
        self.is_streaming = False

    def _stream_request(self, messages):
        """內部流式請求實現"""
        # 這裡是之前的流式請求代碼
        pass

# 使用示例
def on_new_chunk(chunk, full_text):
    """處理新的文本片段"""
    print(chunk, end='', flush=True)

    # 可以在這裡添加其他處理邏輯
    # 比如更新UI、保存到數據庫等

def on_stream_complete(full_response):
    """流式響應完成時的處理"""
    print(f"\n\n回覆完成，總長度：{len(full_response)}字符")

    # 可以在這裡保存完整回覆、更新狀態等

# 創建聊天機器人
bot = StreamingChatBot("Bearer 你的API密鑰")

# 開始流式對話
messages = [{"role": "user", "content": "寫一首關於春天的詩"}]
thread = bot.stream_with_callback(
    messages,
    on_chunk=on_new_chunk,
    on_complete=on_stream_complete
)

# 如果需要，可以中途停止
# time.sleep(5)
# bot.stop_streaming()

# 等待完成
thread.join()
```

進階流式**互動**響應處理：

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
WeMust AI API 互動式流式處理示例
這個示例展示如何創建一個真正互動式的流式對話體驗
構建你的第一個聊天機器人吧！
"""
```

```python
import requests
import json
import threading
import time
import sys
import os
from queue import Queue, Empty
import signal

class InteractiveStreamingChat:
    def __init__(self, api_key, model="gpt-4o-mini"):
        self.api_key = api_key
        self.model = model
        self.base_url = "https://ai-
apigateway.must.edu.mo/openhub/v1/chat/completions"
        self.conversation_history = []
        self.is_streaming = False
        self.current_response = ""
        self.input_queue = Queue()
        self.stop_event = threading.Event()

    def set_system_prompt(self, prompt):
        """設置系統提示"""
        self.conversation_history = [
            {"role": "system", "content": prompt}
        ]

    def stream_response(self, user_input):
        """流式獲取AI回覆"""
        # 添加用戶輸入到歷史
        self.conversation_history.append({
            "role": "user",
            "content": user_input
        })

        headers = {
            "Authorization": self.api_key,
            "Content-Type": "application/json"
        }

        data = {
            "model": self.model,
            "messages": self.conversation_history,
            "stream": True,
            "temperature": 0.7,
            "max_tokens": 1000
        }

        try:
            response = requests.post(
                self.base_url,
                headers=headers,
                json=data,
```

```
                stream=True,
                timeout=30
            )

            if response.status_code == 200:
                self.current_response = ""
                self.is_streaming = True

                print(f"\n🤖 AI: ", end="", flush=True)

                for line in response.iter_lines():
                    if self.stop_event.is_set():
                        break

                    if line:
                        line = line.decode('utf-8').strip()

                        if line.startswith('data:'):
                            data_content = line.split('data:', 1)[1].strip()

                            if data_content == '[DONE]':
                                break

                            try:
                                chunk = json.loads(data_content)

                                if 'choices' in chunk and len(chunk['choices']) >
0:
                                    delta = chunk['choices'][0].get('delta', {})
                                    if 'content' in delta:
                                        content = delta['content']
                                        print(content, end='', flush=True)
                                        self.current_response += content

                                        # 模擬打字機效果
                                        time.sleep(0.02)

                            except json.JSONDecodeError:
                                continue

                # 添加完整回覆到歷史
                if self.current_response:
                    self.conversation_history.append({
                        "role": "assistant",
                        "content": self.current_response
                    })

                print("\n")  # 換行
                self.is_streaming = False
                return True

            else:
                print(f"\n❌ API錯誤：{response.status_code} - {response.text}")
                return False
```

```
        except Exception as e:
            print(f"\n❌ 請求失敗：{e}")
            return False

    def handle_user_input(self):
        """處理用戶輸入的線程"""
        while not self.stop_event.is_set():
            try:
                if sys.stdin in select.select([sys.stdin], [], [], 0.1)[0]:
                    user_input = input().strip()
                    if user_input:
                        self.input_queue.put(user_input)
            except:
                # 在Windows上select可能不可用，使用替代方案
                try:
                    user_input = input().strip()
                    if user_input:
                        self.input_queue.put(user_input)
                except EOFError:
                    break

    def interactive_chat(self):
        """開始互動式聊天"""
        print("🎉 歡迎使用WeMust AI互動式流式聊天！")
        print("💡 特殊命令：")
        print("   - 輸入 '/stop' 中斷當前AI回覆")
        print("   - 輸入 '/clear' 清除對話歷史")
        print("   - 輸入 '/save' 保存對話")
        print("   - 輸入 '/quit' 退出程序")
        print("   - 在AI回覆過程中按Enter可以中斷")
        print("-" * 50)

        # 設置信號處理
        signal.signal(signal.SIGINT, self.signal_handler)

        while not self.stop_event.is_set():
            try:
                # 獲取用戶輸入
                user_input = input("\n👤 你: ").strip()

                if not user_input:
                    continue

                # 處理特殊命令
                if user_input.lower() == '/quit':
                    print("👋 再見！")
                    break
                elif user_input.lower() == '/clear':
                    self.clear_conversation()
                    continue
                elif user_input.lower() == '/save':
                    self.save_conversation()
                    continue
```

```
                elif user_input.lower() == '/stop':
                    if self.is_streaming:
                        self.stop_current_stream()
                    else:
                        print("💭 當前沒有進行中的回覆")
                    continue

                # 開始流式回覆
                print("🔄 AI正在思考...")

                # 在新線程中處理流式回覆
                stream_thread = threading.Thread(
                    target=self.stream_response,
                    args=(user_input,)
                )
                stream_thread.daemon = True
                stream_thread.start()

                # 監聽用戶中斷
                self.monitor_interruption(stream_thread)

            except KeyboardInterrupt:
                self.handle_interrupt()
            except EOFError:
                print("\n👋 再見！")
                break

    def monitor_interruption(self, stream_thread):
        """監聽用戶中斷"""
        while stream_thread.is_alive() and self.is_streaming:
            try:
                # 檢查是否有新的輸入（中斷信號）
                import select
                if sys.stdin in select.select([sys.stdin], [], [], 0.1)[0]:
                    interrupt_input = sys.stdin.readline().strip()
                    if interrupt_input == "" or interrupt_input.lower() in
['/stop', 'stop']:
                        print("\n⏸️  用戶中斷回覆")
                        self.stop_current_stream()
                        break
            except:
                # Windows兼容性處理
                time.sleep(0.1)

        stream_thread.join(timeout=1)

    def stop_current_stream(self):
        """停止當前的流式回覆"""
        self.stop_event.set()
        self.is_streaming = False
        print("\n⏹️  已停止當前回覆")
        time.sleep(0.5)
        self.stop_event.clear()
```

```
    def clear_conversation(self):
        """清除對話歷史"""
        system_messages = [msg for msg in self.conversation_history if msg["role"]
== "system"]
        self.conversation_history = system_messages
        print("🗑️ 對話歷史已清除")

    def save_conversation(self):
        """保存對話歷史"""
        timestamp = time.strftime("%Y%m%d_%H%M%S")
        filename = f"chat_history_{timestamp}.json"

        try:
            with open(filename, 'w', encoding='utf-8') as f:
                json.dump(self.conversation_history, f, ensure_ascii=False,
indent=2)
            print(f"💾 對話已保存到: {filename}")
        except Exception as e:
            print(f"❌ 保存失敗: {e}")

    def signal_handler(self, signum, frame):
        """處理Ctrl+C信號"""
        if self.is_streaming:
            self.stop_current_stream()
        else:
            print("\n👋 再見！")
            sys.exit(0)

    def handle_interrupt(self):
        """處理鍵盤中斷"""
        if self.is_streaming:
            self.stop_current_stream()
        else:
            print("\n👋 再見！")
            sys.exit(0)

class EnhancedStreamingChat(InteractiveStreamingChat):
    """增強版互動式流式聊天，支持更多功能"""

    def __init__(self, api_key, model="gpt-4o-mini"):
        super().__init__(api_key, model)
        self.typing_speed = 0.02  # 打字機效果速度
        self.show_thinking = True  # 是否顯示思考過程
        self.auto_save = True     # 是否自動保存

    def set_typing_speed(self, speed):
        """設置打字機效果速度"""
        self.typing_speed = max(0, min(0.1, speed))
        print(f"⚡ 打字速度設置為: {speed}秒/字符")

    def toggle_thinking_display(self):
        """切換思考過程顯示"""
        self.show_thinking = not self.show_thinking
        status = "開啟" if self.show_thinking else "關閉"
```

```
        print(f"🤔 思考過程顯示已{status}")

    def stream_response(self, user_input):
        """增強版流式回覆"""
        # 添加用戶輸入到歷史
        self.conversation_history.append({
            "role": "user",
            "content": user_input
        })

        headers = {
            "Authorization": self.api_key,
            "Content-Type": "application/json"
        }

        data = {
            "model": self.model,
            "messages": self.conversation_history,
            "stream": True,
            "temperature": 0.7,
            "max_tokens": 1000
        }

        try:
            if self.show_thinking:
                print("🤔 AI正在分析你的問題...")
                time.sleep(0.5)
                print("💭 正在組織回覆...")
                time.sleep(0.3)

            response = requests.post(
                self.base_url,
                headers=headers,
                json=data,
                stream=True,
                timeout=30
            )

            if response.status_code == 200:
                self.current_response = ""
                self.is_streaming = True

                print(f"\n🤖 AI: ", end="", flush=True)

                word_buffer = ""

                for line in response.iter_lines():
                    if self.stop_event.is_set():
                        break

                    if line:
                        line = line.decode('utf-8').strip()

                        if line.startswith('data:'):
```

```
                            data_content = line.split('data:', 1)[1].strip()

                            if data_content == '[DONE]':
                                break

                            try:
                                chunk = json.loads(data_content)

                                if 'choices' in chunk and len(chunk['choices']) >
0:
                                    delta = chunk['choices'][0].get('delta', {})
                                    if 'content' in delta:
                                        content = delta['content']

                                        # 實現更自然的打字效果
                                        for char in content:
                                            if self.stop_event.is_set():
                                                break
                                            print(char, end='', flush=True)
                                            self.current_response += char

                                            # 在標點符號後稍作停頓
                                            if char in '，。！？；：':
                                                time.sleep(self.typing_speed * 3)
                                            else:
                                                time.sleep(self.typing_speed)

                            except json.JSONDecodeError:
                                continue

                    # 添加完整回覆到歷史
                    if self.current_response:
                        self.conversation_history.append({
                            "role": "assistant",
                            "content": self.current_response
                        })

                        # 自動保存
                        if self.auto_save and len(self.conversation_history) % 10 ==
0:
                            self.auto_save_conversation()

                    print("\n")  # 換行
                    self.is_streaming = False
                    return True

                else:
                    print(f"\n❌ API錯誤：{response.status_code} - {response.text}")
                    return False

            except Exception as e:
                print(f"\n❌ 請求失敗：{e}")
                return False
```

```
    def auto_save_conversation(self):
        """自動保存對話"""
        timestamp = time.strftime("%Y%m%d_%H%M%S")
        filename = f"auto_save_{timestamp}.json"

        try:
            with open(filename, 'w', encoding='utf-8') as f:
                json.dump(self.conversation_history, f, ensure_ascii=False,
indent=2)
            print(f"\n💾 [自動保存] {filename}")
        except:
            pass  # 靜默失敗

    def interactive_chat(self):
        """增強版互動式聊天"""
        print("🎉 歡迎使用WeMust AI增強版互動式流式聊天！")
        print("💡 特殊命令：")
        print("   - '/stop' - 中斷當前AI回覆")
        print("   - '/clear' - 清除對話歷史")
        print("   - '/save' - 保存對話")
        print("   - '/speed <數字>' - 設置打字速度(0-0.1)")
        print("   - '/thinking' - 切換思考過程顯示")
        print("   - '/stats' - 顯示對話統計")
        print("   - '/help' - 顯示幫助")
        print("   - '/quit' - 退出程序")
        print("-" * 50)

        # 設置信號處理
        signal.signal(signal.SIGINT, self.signal_handler)

        while not self.stop_event.is_set():
            try:
                # 獲取用戶輸入
                user_input = input("\n👤 你: ").strip()

                if not user_input:
                    continue

                # 處理特殊命令
                if user_input.lower() == '/quit':
                    print("👋 再見！")
                    break
                elif user_input.lower() == '/clear':
                    self.clear_conversation()
                    continue
                elif user_input.lower() == '/save':
                    self.save_conversation()
                    continue
                elif user_input.lower() == '/thinking':
                    self.toggle_thinking_display()
                    continue
                elif user_input.lower() == '/stats':
                    self.show_stats()
                    continue
```

```
                elif user_input.lower() == '/help':
                    self.show_help()
                    continue
                elif user_input.lower().startswith('/speed'):
                    try:
                        speed = float(user_input.split()[1])
                        self.set_typing_speed(speed)
                    except:
                        print("❌ 請輸入有效的速度值，例如: /speed 0.05")
                    continue
                elif user_input.lower() == '/stop':
                    if self.is_streaming:
                        self.stop_current_stream()
                    else:
                        print("💭 當前沒有進行中的回覆")
                    continue

                # 開始流式回覆
                stream_thread = threading.Thread(
                    target=self.stream_response,
                    args=(user_input,)
                )
                stream_thread.daemon = True
                stream_thread.start()

                # 監聽用戶中斷
                self.monitor_interruption(stream_thread)

            except KeyboardInterrupt:
                self.handle_interrupt()
            except EOFError:
                print("\n👋 再見！")
                break

    def show_stats(self):
        """顯示對話統計"""
        total_messages = len(self.conversation_history)
        user_messages = len([msg for msg in self.conversation_history if
msg["role"] == "user"])
        ai_messages = len([msg for msg in self.conversation_history if msg["role"]
== "assistant"])

        print(f"📊 對話統計:")
        print(f"   總消息數: {total_messages}")
        print(f"   用戶消息: {user_messages}")
        print(f"   AI回覆: {ai_messages}")
        print(f"   使用模型: {self.model}")

    def show_help(self):
        """顯示幫助信息"""
        print("📖 幫助信息:")
        print("   這是一個互動式的AI聊天程序，支持實時流式回覆")
        print("   你可以隨時中斷AI的回覆，調整設置，保存對話等")
        print("   輸入 '/quit' 退出程序")
```

```python
def main():
    """主程序"""
    print("🚀 WeMust AI 互動式流式聊天啟動器")
    print("請選擇聊天模式:")
    print("1. 基礎互動模式")
    print("2. 增強互動模式")

    try:
        choice = input("請選擇 (1/2): ").strip()

        api_key = input("請輸入你的API密鑰: ").strip()
        if not api_key:
            print("❌ API密鑰不能為空！")
            return

        # 選擇模型
        print("\n可用模型:")
        models = [
            "gpt-4o-mini",
            "glm-4-flash",
            "glm-4"
        ]

        for i, model in enumerate(models, 1):
            print(f"{i}. {model}")

        try:
            model_choice = int(input("請選擇模型 (1-4): ")) - 1
            selected_model = models[model_choice]
        except:
            selected_model = "gpt-4o-mini"
            print(f"使用默認模型: {selected_model}")

        # 創建聊天實例
        if choice == "2":
            chat = EnhancedStreamingChat(api_key, selected_model)
        else:
            chat = InteractiveStreamingChat(api_key, selected_model)

        # 設置系統角色
        system_prompt = input("設置AI角色 (直接回車使用默認): ").strip()
        if not system_prompt:
            system_prompt = "你是一個友善、有幫助的AI助手，喜歡用生動的語言與用戶交流"

        chat.set_system_prompt(system_prompt)
        print(f"✅ AI角色已設置: {system_prompt}")

        # 開始聊天
        chat.interactive_chat()

    except KeyboardInterrupt:
        print("\n👋 程序已退出")
    except Exception as e:
```

```
        print(f"❌ 程序錯誤: {e}")

if __name__ == "__main__":
    # 導入必要的模塊
    try:
        import select
    except ImportError:
        # Windows兼容性
        select = None

    main()
```

3.3 工具調用功能

工具調用（Function Calling）是WeMust AI API的高級功能，它允許AI調用外部函數來獲取實時信息或執行特定操作。這大大擴展了AI的能力範圍。

3.3.1 工具調用概念

工具調用讓AI能夠：

- 獲取實時數據（如天氣、股價、新聞）
- 執行計算任務
- 與外部系統交互
- 調用自定義業務邏輯

工具調用的工作流程：

```
用戶提問 → AI分析需求 → 決定調用工具 → 執行工具函數 → 獲取結果 → 生成最終回覆
```

3.3.2 自定義函數集成

基本工具定義：

```
# 定義一個獲取天氣的工具
weather_tool = {
    "type": "function",
    "function": {
        "name": "get_current_weather",
        "description": "獲取指定城市的當前天氣信息",
        "parameters": {
            "type": "object",
            "properties": {
                "location": {
                    "type": "string",
                    "description": "城市名稱，例如：台北、香港"
                },
                "unit": {
```

```
                        "type": "string",
                        "enum": ["celsius", "fahrenheit"],
                        "description": "溫度單位"
                    }
                },
                "required": ["location"]
            }
        }
    }

    # 定義一個計算器工具
    calculator_tool = {
        "type": "function",
        "function": {
            "name": "calculate",
            "description": "執行數學計算",
            "parameters": {
                "type": "object",
                "properties": {
                    "expression": {
                        "type": "string",
                        "description": "數學表達式，例如：2+3*4"
                    }
                },
                "required": ["expression"]
            }
        }
    }
```

實現工具函數：

```
import json
import requests
from datetime import datetime

def get_current_weather(location, unit="celsius"):
    """
    獲取天氣信息的實際實現
    這裡使用模擬數據，實際應用中可以調用真實的天氣API
    """
    # 模擬天氣數據
    weather_data = {
        "台北": {"temperature": 25, "condition": "晴朗", "humidity": 60},
        "香港": {"temperature": 28, "condition": "多雲", "humidity": 75},
        "上海": {"temperature": 22, "condition": "小雨", "humidity": 80}
    }

    if location in weather_data:
        data = weather_data[location]
        temp = data["temperature"]

        if unit == "fahrenheit":
```

```
            temp = temp * 9/5 + 32

        return {
            "location": location,
            "temperature": temp,
            "unit": unit,
            "condition": data["condition"],
            "humidity": data["humidity"],
            "timestamp": datetime.now().isoformat()
        }
    else:
        return {"error": f"無法獲取{location}的天氣信息"}

def calculate(expression):
    """
    安全的數學計算函數
    """
    try:
        # 只允許基本的數學運算
        allowed_chars = set('0123456789+-*/.() ')
        if not all(c in allowed_chars for c in expression):
            return {"error": "表達式包含不允許的字符"}

        result = eval(expression)
        return {"expression": expression, "result": result}
    except Exception as e:
        return {"error": f"計算錯誤：{str(e)}"}

# 工具函數映射
available_functions = {
    "get_current_weather": get_current_weather,
    "calculate": calculate
}
```

完整的工具調用實現：

```
def chat_with_tools(messages, tools=None):
    """
    支持工具調用的聊天函數
    """
    api_key = "Bearer 你的API密鑰"

    headers = {
        "Authorization": api_key,
        "Content-Type": "application/json"
    }

    data = {
        "model": "gpt-4o-mini",
        "messages": messages,
        "temperature": 0.7
    }
```

```
    # 如果提供了工具，添加到請求中
    if tools:
        data["tools"] = tools
        data["tool_choice"] = "auto"

    try:
        response = requests.post(
            "https://ai-apigateway.must.edu.mo/openhub/v1/chat/completions",
            headers=headers,
            json=data
        )

        if response.status_code == 200:
            result = response.json()
            message = result["choices"][0]["message"]

            # 檢查是否需要調用工具
            if message.get("tool_calls"):
                return handle_tool_calls(message, messages, tools)
            else:
                return message["content"]

        else:
            return f"錯誤：{response.status_code} - {response.text}"

    except Exception as e:
        return f"請求失敗：{e}"

def handle_tool_calls(assistant_message, messages, tools):
    """
    處理工具調用
    """
    # 將助手的工具調用消息添加到對話歷史
    messages.append(assistant_message)

    # 執行每個工具調用
    for tool_call in assistant_message["tool_calls"]:
        function_name = tool_call["function"]["name"]
        function_args = json.loads(tool_call["function"]["arguments"])

        # 調用對應的函數
        if function_name in available_functions:
            function_result = available_functions[function_name](**function_args)
        else:
            function_result = {"error": f"未知函數：{function_name}"}

        # 將工具調用結果添加到對話歷史
        messages.append({
            "role": "tool",
            "tool_call_id": tool_call["id"],
            "content": json.dumps(function_result, ensure_ascii=False)
        })
```

```
    # 再次調用API獲取最終回覆
    return chat_with_tools(messages, tools)

# 使用示例
tools = [weather_tool, calculator_tool]

messages = [
    {
        "role": "system",
        "content": "你是一個智能助手，可以幫助用戶獲取天氣信息和進行數學計算"
    },
    {
        "role": "user",
        "content": "台北今天天氣如何？另外幫我算一下 15 * 8 + 32"
    }
]

response = chat_with_tools(messages, tools)
print("AI回覆：", response)
```

以下是完整代碼：

```
import json
import requests
from datetime import datetime

weather_tool = {
    "type": "function",
    "function": {
        "name": "get_current_weather",
        "description": "獲取指定城市的當前天氣信息",
        "parameters": {
            "type": "object",
            "properties": {
                "location": {
                    "type": "string",
                    "description": "城市名稱，例如：台北、香港"
                },
                "unit": {
                    "type": "string",
                    "enum": ["celsius", "fahrenheit"],
                    "description": "溫度單位"
                }
            },
            "required": ["location"]
        }
    }
}

# 定義一個計算器工具
calculator_tool = {
    "type": "function",
```

```
        "function": {
            "name": "calculate",
            "description": "執行數學計算",
            "parameters": {
                "type": "object",
                "properties": {
                    "expression": {
                        "type": "string",
                        "description": "數學表達式，例如：2+3*4"
                    }
                },
                "required": ["expression"]
            }
        }
    }

    def get_current_weather(location, unit="celsius"):
        """
        獲取天氣信息的實際實現
        這裡使用模擬數據，實際應用中可以調用真實的天氣API
        """
        # 模擬天氣數據
        weather_data = {
            "台北": {"temperature": 25, "condition": "晴朗", "humidity": 60},
            "香港": {"temperature": 28, "condition": "多雲", "humidity": 75},
            "上海": {"temperature": 22, "condition": "小雨", "humidity": 80}
        }

        if location in weather_data:
            data = weather_data[location]
            temp = data["temperature"]

            if unit == "fahrenheit":
                temp = temp * 9/5 + 32

            return {
                "location": location,
                "temperature": temp,
                "unit": unit,
                "condition": data["condition"],
                "humidity": data["humidity"],
                "timestamp": datetime.now().isoformat()
            }
        else:
            return {"error": f"無法獲取{location}的天氣信息"}

    def calculate(expression):
        """
        安全的數學計算函數
        """
        try:
            # 只允許基本的數學運算
            allowed_chars = set('0123456789+-*/.() ')
            if not all(c in allowed_chars for c in expression):
```

```
            return {"error": "表達式包含不允許的字符"}

        result = eval(expression)
        return {"expression": expression, "result": result}
    except Exception as e:
        return {"error": f"計算錯誤：{str(e)}"}

# 工具函數映射
available_functions = {
    "get_current_weather": get_current_weather,
    "calculate": calculate
}
def chat_with_tools(messages, tools=None):
    """
    支持工具調用的聊天函數
    """
    api_key = "你的API密鑰"

    headers = {
        "Authorization": api_key,
        "Content-Type": "application/json"
    }

    data = {
        "model": "gpt-4o-mini",
        "messages": messages,
        "temperature": 0.7
    }

    # 如果提供了工具，添加到請求中
    if tools:
        data["tools"] = tools
        data["tool_choice"] = "auto"

    try:
        response = requests.post(
            "https://ai-apigateway.must.edu.mo/openhub/v1/chat/completions",
            headers=headers,
            json=data
        )

        if response.status_code == 200:
            result = response.json()
            message = result["choices"][0]["message"]

            # 檢查是否需要調用工具
            if message.get("tool_calls"):
                return handle_tool_calls(message, messages, tools)
            else:
                return message["content"]

        else:
            return f"錯誤：{response.status_code} - {response.text}"
```

```python
    except Exception as e:
        return f"請求失敗：{e}"

def handle_tool_calls(assistant_message, messages, tools):
    """
    處理工具調用
    """
    # 將助手的工具調用消息添加到對話歷史
    messages.append(assistant_message)

    # 執行每個工具調用
    for tool_call in assistant_message["tool_calls"]:
        function_name = tool_call["function"]["name"]
        function_args = json.loads(tool_call["function"]["arguments"])

        # 調用對應的函數
        if function_name in available_functions:
            function_result = available_functions[function_name](**function_args)
        else:
            function_result = {"error": f"未知函數：{function_name}"}

        # 將工具調用結果添加到對話歷史
        messages.append({
            "role": "tool",
            "tool_call_id": tool_call["id"],
            "content": json.dumps(function_result, ensure_ascii=False)
        })

    # 再次調用API獲取最終回覆
    return chat_with_tools(messages, tools)

# 使用示例
tools = [weather_tool, calculator_tool]

messages = [
    {
        "role": "system",
        "content": "你是一個智能助手，可以幫助用戶獲取天氣信息和進行數學計算"
    },
    {
        "role": "user",
        "content": "台北今天天氣如何？另外幫我算一下 15 * 8 + 32"
    }
]

response = chat_with_tools(messages, tools)
print("AI回覆：", response)
```

通過掌握這些核心功能，你已經能夠構建功能豐富的AI應用了。接下來的章節將教你如何優化參數以獲得最佳效果。

第四章：參數調優指南

4.1 核心參數詳解

理解和掌握API參數的調優是獲得最佳AI效果的關鍵。每個參數都會影響AI的行為和輸出質量，學會合理調整這些參數能讓你的應用表現更出色。

4.1.1 temperature（創意度控制）

temperature參數控制AI回覆的隨機性和創意程度，是最重要的調優參數之一。

參數範圍：0.0 - 2.0 **默認值**：1.0

參數效果說明：

- **0.0 - 0.3（低溫度）**：
  - 回覆非常確定和一致
  - 適合事實查詢、數學計算
  - 每次運行結果基本相同
  - 創意性較低
- **0.4 - 0.7（中等溫度）**：
  - 平衡確定性和創意性
  - 適合大多數日常對話
  - 回覆有一定變化但保持合理
  - 推薦的通用設置
- **0.8 - 1.2（高溫度）**：
  - 更有創意和多樣性
  - 適合創意寫作、頭腦風暴
  - 回覆更加豐富多彩
  - 可能出現意外的創新想法
- **1.3 - 2.0（極高溫度）**：
  - 極高的隨機性
  - 可能產生不連貫的內容
  - 適合實驗性創作
  - 需要謹慎使用

實際效果對比：

```
# 測試不同temperature的效果
def test_temperature_effects():
    base_message = [
        {"role": "system", "content": "你是一個詩人"},
        {"role": "user", "content": "寫一首關於月亮的短詩"}
    ]
```

```
    temperatures = [0.1, 0.5, 0.9, 1.5]

    for temp in temperatures:
        print(f"\n=== Temperature: {temp} ===")
        response = chat_with_ai(base_message, temperature=temp)
        print(response)
        print("-" * 50)

# 不同場景的temperature建議
temperature_guide = {
    "事實查詢": {
        "temperature": 0.1,
        "說明": "需要準確、一致的答案",
        "示例": "什麼是光速？"
    },
    "學術寫作": {
        "temperature": 0.3,
        "說明": "保持專業性和準確性",
        "示例": "解釋量子力學的基本原理"
    },
    "日常對話": {
        "temperature": 0.7,
        "說明": "自然、友好的交流",
        "示例": "今天天氣真好，你覺得呢？"
    },
    "創意寫作": {
        "temperature": 0.9,
        "說明": "鼓勵創新和想像力",
        "示例": "寫一個科幻短故事"
    },
    "頭腦風暴": {
        "temperature": 1.2,
        "說明": "產生多樣化的想法",
        "示例": "為新產品想10個創意名稱"
    }
}
```

4.1.2 max_tokens（回覆長度控制）

max_tokens參數限制AI回覆的最大長度，幫助控制成本和回覆的簡潔性。

重要概念：

- 1個token ≈ 0.75個英文單詞
- 1個中文字符 ≈ 1-2個tokens
- 包括輸入和輸出的總token數

長度規劃指南：

```python
# 不同內容類型的token估算
content_token_guide = {
    "簡短回答": {
        "max_tokens": 50,
        "適用場景": "是/否問題、簡單解釋",
        "中文字數": "約25-40字",
        "示例": "什麼是API？"
    },
    "標準回答": {
        "max_tokens": 200,
        "適用場景": "一般問答、概念解釋",
        "中文字數": "約100-150字",
        "示例": "解釋機器學習的基本概念"
    },
    "詳細說明": {
        "max_tokens": 500,
        "適用場景": "深入解釋、教學內容",
        "中文字數": "約250-400字",
        "示例": "如何學習編程？"
    },
    "長篇內容": {
        "max_tokens": 1000,
        "適用場景": "文章、故事、詳細分析",
        "中文字數": "約500-800字",
        "示例": "寫一篇關於AI發展的文章"
    },
    "超長內容": {
        "max_tokens": 2000,
        "適用場景": "長篇故事、深度分析",
        "中文字數": "約1000-1600字",
        "示例": "創作一個完整的短篇小說"
    }
}

def estimate_tokens(text):
    """
    估算文本的token數量
    這是一個簡化的估算方法
    """
    # 中文字符
    chinese_chars = len([c for c in text if '\u4e00' <= c <= '\u9fff'])
    # 英文單詞（簡化計算）
    english_words = len(text.split()) - chinese_chars

    # 估算token數
    estimated_tokens = chinese_chars * 1.5 + english_words * 1.3
    return int(estimated_tokens)

# 動態調整max_tokens
def adaptive_max_tokens(user_input, response_type="standard"):
    """
    根據用戶輸入和響應類型動態調整max_tokens
    """
```

```
    input_tokens = estimate_tokens(user_input)

    multipliers = {
        "brief": 0.5,       # 簡短回答
        "standard": 2.0,    # 標準回答
        "detailed": 4.0,    # 詳細回答
        "extensive": 8.0    # 擴展回答
    }

    base_tokens = max(100, input_tokens * multipliers.get(response_type, 2.0))
    return min(base_tokens, 2000)  # 限制最大值
```

4.1.3 top_p（多樣性控制）

top_p參數（也稱為nucleus sampling）控制AI選擇詞彙的範圍，影響回覆的多樣性和一致性。

參數範圍：0.0 - 1.0 **默認值**：1.0

工作原理：

- AI會計算每個可能詞彙的概率
- top_p決定從概率最高的詞彙中選擇多少比例
- 較小的值讓AI選擇更保守
- 較大的值增加詞彙選擇的多樣性

參數效果：

```
# top_p效果示例
top_p_guide = {
    0.1: {
        "效果": "極度保守，選擇最可能的詞彙",
        "適用": "需要高度一致性的場景",
        "風險": "可能過於重複和單調"
    },
    0.3: {
        "效果": "較為保守，但有一定變化",
        "適用": "技術文檔、正式回覆",
        "風險": "創意性可能不足"
    },
    0.7: {
        "效果": "平衡的多樣性和一致性",
        "適用": "大多數日常對話",
        "風險": "通用設置，風險較低"
    },
    0.9: {
        "效果": "高多樣性，更多詞彙選擇",
        "適用": "創意寫作、頭腦風暴",
        "風險": "可能出現意外的詞彙組合"
    },
    1.0: {
        "效果": "最大多樣性，考慮所有可能",
```

```
        "適用": "實驗性創作",
        "風險": "可能產生不連貫的內容"
    }
}

# temperature和top_p的組合效果
def get_parameter_combination(scenario):
    """
    根據場景推薦temperature和top_p的組合
    """
    combinations = {
        "精確回答": {
            "temperature": 0.1,
            "top_p": 0.3,
            "說明": "最大化準確性和一致性"
        },
        "專業寫作": {
            "temperature": 0.3,
            "top_p": 0.7,
            "說明": "保持專業性但允許適度變化"
        },
        "友好對話": {
            "temperature": 0.7,
            "top_p": 0.9,
            "說明": "自然、多樣的對話風格"
        },
        "創意創作": {
            "temperature": 0.9,
            "top_p": 0.9,
            "說明": "最大化創意和多樣性"
        },
        "實驗性": {
            "temperature": 1.2,
            "top_p": 1.0,
            "說明": "探索極限創意可能性"
        }
    }

    return combinations.get(scenario, combinations["友好對話"])
```

4.2 不同場景的參數配置

根據具體的應用場景選擇合適的參數組合，能夠顯著提升AI的表現效果。

4.2.1 學術問答場景

學術問答需要準確、權威、結構化的回覆，重點是事實的正確性和邏輯的清晰性。

推薦參數配置：

```
academic_config = {
    "temperature": 0.2,
    "top_p": 0.8,
    "max_tokens": 800,
    "frequency_penalty": 0.0,
    "presence_penalty": 0.0
}

# 學術問答的系統提示
academic_system_prompt = """
你是一位博學的學術助手，具有以下特點：
1. 回答基於可靠的學術資料和研究
2. 提供準確、客觀的信息
3. 承認知識的局限性，不確定時會說明
4. 使用清晰的邏輯結構組織回答
5. 適當引用相關的理論和概念
6. 避免主觀臆測和不確定的陳述
"""

def academic_chat(question):
    """學術問答專用函數"""
    messages = [
        {"role": "system", "content": academic_system_prompt},
        {"role": "user", "content": question}
    ]

    return chat_with_ai(messages, **academic_config)

# 使用示例
academic_questions = [
    "解釋量子糾纏的基本原理",
    "什麼是深度學習中的反向傳播算法？",
    "分析莎士比亞作品中的人文主義思想"
]

for question in academic_questions:
    print(f"問題：{question}")
    answer = academic_chat(question)
    print(f"回答：{answer}\n")
```

4.2.2 創意寫作場景

創意寫作需要豐富的想像力、多樣的表達方式和引人入勝的內容。

推薦參數配置：

```
creative_config = {
    "temperature": 0.8,
    "top_p": 0.9,
    "max_tokens": 1500,
```

```
    "frequency_penalty": 0.3,  # 減少重複
    "presence_penalty": 0.3    # 鼓勵新話題
}

# 創意寫作的系統提示
creative_system_prompt = """
你是一位富有想像力的創意作家，具有以下特點：
1. 擅長創造生動、引人入勝的故事
2. 運用豐富的修辭手法和文學技巧
3. 能夠創造獨特的角色和情節
4. 注重情感表達和氛圍營造
5. 勇於嘗試新穎的創意和視角
6. 語言優美，富有感染力
"""

def creative_writing(prompt, style="故事"):
    """創意寫作專用函數"""
    enhanced_prompt = f"請創作一個{style}：{prompt}"

    messages = [
        {"role": "system", "content": creative_system_prompt},
        {"role": "user", "content": enhanced_prompt}
    ]

    return chat_with_ai(messages, **creative_config)

# 創意寫作示例
creative_prompts = [
    ("一隻會說話的貓咪的冒險", "短篇故事"),
    ("未來世界的一天", "科幻小說"),
    ("春天的第一場雨", "散文詩"),
    ("時間旅行者的困境", "懸疑故事")
]

for prompt, style in creative_prompts:
    print(f"創作主題：{prompt}（{style}）")
    story = creative_writing(prompt, style)
    print(f"作品：\n{story}\n")
    print("=" * 60)
```

4.2.3 代碼生成場景

代碼生成需要精確性、邏輯性和可執行性，同時要考慮代碼的可讀性和最佳實踐。

推薦參數配置：

```
coding_config = {
    "temperature": 0.1,
    "top_p": 0.95,
    "max_tokens": 1200,
    "frequency_penalty": 0.0,
```

```python
    "presence_penalty": 0.0
}

# 代碼生成的系統提示
coding_system_prompt = """
你是一位經驗豐富的程序員，具有以下特點：
1. 編寫清晰、可讀、高效的代碼
2. 遵循編程最佳實踐和代碼規範
3. 提供詳細的代碼註釋和說明
4. 考慮錯誤處理和邊界情況
5. 推薦合適的算法和數據結構
6. 提供完整可運行的代碼示例
7. 解釋代碼的工作原理和設計思路
"""

def generate_code(requirement, language="Python"):
    """代碼生成專用函數"""
    enhanced_prompt = f"""
    請用{language}實現以下需求：{requirement}

    要求：
    1. 提供完整可運行的代碼
    2. 添加詳細的註釋說明
    3. 包含錯誤處理
    4. 提供使用示例
    5. 解釋關鍵算法和邏輯
    """

    messages = [
        {"role": "system", "content": coding_system_prompt},
        {"role": "user", "content": enhanced_prompt}
    ]

    return chat_with_ai(messages, **coding_config)

# 代碼生成示例
coding_tasks = [
    ("實現一個二分查找算法", "Python"),
    ("創建一個簡單的計算器類", "Python"),
    ("編寫一個文件讀寫工具", "Python"),
    ("實現一個基本的排序算法比較", "Python")
]

for task, lang in coding_tasks:
    print(f"編程任務：{task}（{lang}）")
    code = generate_code(task, lang)
    print(f"生成的代碼：\n{code}\n")
    print("=" * 80)
```

4.3 參數調優實戰練習

通過實際練習來掌握參數調優的技巧，這是提升AI應用效果的最佳方法。

練習1：對比測試不同參數組合

```python
def parameter_comparison_test():
    """
    對比測試不同參數組合的效果
    """
    test_prompt = "請解釋什麼是人工智能"

    # 定義不同的參數組合
    parameter_sets = {
        "保守型": {"temperature": 0.2, "top_p": 0.7, "max_tokens": 300},
        "平衡型": {"temperature": 0.7, "top_p": 0.9, "max_tokens": 300},
        "創意型": {"temperature": 1.0, "top_p": 0.95, "max_tokens": 300},
        "極端型": {"temperature": 1.5, "top_p": 1.0, "max_tokens": 300}
    }

    messages = [
        {"role": "system", "content": "你是一個AI助手"},
        {"role": "user", "content": test_prompt}
    ]

    results = {}

    for name, params in parameter_sets.items():
        print(f"\n=== {name}參數組合 ===")
        print(f"參數：{params}")

        # 多次測試以觀察一致性
        responses = []
        for i in range(3):
            response = chat_with_ai(messages, **params)
            responses.append(response)
            print(f"第{i+1}次回覆：{response[:100]}...")

        results[name] = responses

        # 分析一致性
        consistency = analyze_consistency(responses)
        print(f"一致性評分：{consistency}/10")

    return results

def analyze_consistency(responses):
    """
    簡單的一致性分析
    """
    if len(responses) < 2:
        return 10

    # 計算回覆長度的變化
    lengths = [len(r) for r in responses]
    length_variance = max(lengths) - min(lengths)
```

```
    # 計算詞彙重疊度（簡化版）
    words_sets = [set(r.split()) for r in responses]
    common_words = set.intersection(*words_sets)
    total_unique_words = set.union(*words_sets)

    overlap_ratio = len(common_words) / len(total_unique_words) if
total_unique_words else 0

    # 綜合評分（簡化算法）
    consistency_score = (overlap_ratio * 7 + (1 - min(length_variance/100, 1)) *
3)
    return round(consistency_score, 1)
```

練習2：場景化參數優化

```
def scenario_optimization_exercise():
    """
    場景化參數優化練習
    """
    scenarios = {
        "客服回覆": {
            "prompt": "客戶抱怨產品有問題，請回覆",
            "目標": "專業、同理心、解決問題",
            "初始參數": {"temperature": 0.5, "top_p": 0.8}
        },
        "教學解釋": {
            "prompt": "向小學生解釋什麼是重力",
            "目標": "簡單易懂、生動有趣",
            "初始參數": {"temperature": 0.6, "top_p": 0.9}
        },
        "技術文檔": {
            "prompt": "編寫API使用說明",
            "目標": "準確、詳細、結構化",
            "初始參數": {"temperature": 0.2, "top_p": 0.7}
        }
    }

    for scenario_name, scenario_info in scenarios.items():
        print(f"\n=== {scenario_name}場景優化 ===")
        print(f"目標：{scenario_info['目標']}")

        # 測試初始參數
        initial_response = test_scenario_parameters(
            scenario_info["prompt"],
            scenario_info["初始參數"]
        )

        print(f"初始回覆：{initial_response[:200]}...")

        # 嘗試優化參數
        optimized_params = optimize_for_scenario(scenario_name, scenario_info)
```

```
        optimized_response = test_scenario_parameters(
            scenario_info["prompt"],
            optimized_params
        )

        print(f"優化後回覆：{optimized_response[:200]}...")
        print(f"優化參數：{optimized_params}")

def optimize_for_scenario(scenario_name, scenario_info):
    """
    根據場景特點優化參數
    """
    base_params = scenario_info["初始參數"].copy()

    # 根據場景調整參數
    if "客服" in scenario_name:
        # 客服需要一致性和專業性
        base_params.update({
            "temperature": 0.3,
            "top_p": 0.8,
            "frequency_penalty": 0.2
        })
    elif "教學" in scenario_name:
        # 教學需要生動但準確
        base_params.update({
            "temperature": 0.7,
            "top_p": 0.9,
            "presence_penalty": 0.1
        })
    elif "技術" in scenario_name:
        # 技術文檔需要精確性
        base_params.update({
            "temperature": 0.1,
            "top_p": 0.7,
            "frequency_penalty": 0.0
        })

    return base_params
```

練習3：成本效益優化

```
def cost_effectiveness_optimization():
    """
    成本效益優化練習
    """
    test_cases = [
        {
            "task": "簡單問答",
            "prompt": "什麼是Python？",
            "target_length": "簡短"
        },
        {
```

```
            "task": "內容摘要",
            "prompt": "總結一下機器學習的主要應用",
            "target_length": "中等"
        },
        {
            "task": "詳細解釋",
            "prompt": "詳細解釋深度學習的工作原理",
            "target_length": "詳細"
        }
    ]

    # 不同的成本策略
    cost_strategies = {
        "經濟型": {
            "model": "gpt-4o-mini",
            "max_tokens": 200,
            "temperature": 0.5
        },
        "平衡型": {
            "model": "gpt-4o",
            "max_tokens": 500,
            "temperature": 0.7
        },
        "高質量型": {
            "model": "gpt-4-turbo",
            "max_tokens": 1000,
            "temperature": 0.6
        }
    }

    for case in test_cases:
        print(f"\n=== {case['task']} ===")
        print(f"需求：{case['prompt']}")

        for strategy_name, strategy_params in cost_strategies.items():
            print(f"\n{strategy_name}策略：")

            # 調整max_tokens根據目標長度
            adjusted_params = strategy_params.copy()
            if case["target_length"] == "簡短":
                adjusted_params["max_tokens"] = min(adjusted_params["max_tokens"],
150)
            elif case["target_length"] == "詳細":
                adjusted_params["max_tokens"] = max(adjusted_params["max_tokens"],
800)

            response = test_scenario_parameters(case["prompt"], adjusted_params)

            # 估算成本（簡化版）
            estimated_cost = estimate_api_cost(adjusted_params)

            print(f"回覆：{response[:150]}...")
            print(f"估算成本：{estimated_cost}")
            print(f"性價比評分：{evaluate_cost_effectiveness(response,
```

```
estimated_cost)}")

def estimate_api_cost(params):
    """
    估算API調用成本（簡化版）
    """
    model_costs = {
        "gpt-4o-mini": 0.1,
        "gpt-4o": 0.5,
        "gpt-4-turbo": 1.0
    }

    base_cost = model_costs.get(params.get("model", "gpt-4o-mini"), 0.5)
    token_multiplier = params.get("max_tokens", 500) / 500

    return round(base_cost * token_multiplier, 3)

def evaluate_cost_effectiveness(response, cost):
    """
    評估成本效益（簡化版）
    """
    quality_score = min(len(response) / 100, 10)  # 基於長度的質量評分
    cost_efficiency = quality_score / max(cost, 0.1)
    return round(cost_efficiency, 2)
```

通過這些參數調優的學習和練習，你已經掌握了如何根據不同場景優化AI的表現。接下來的章節將通過實際編程項目來鞏固這些知識。

第五章：編程實戰教程

5.1 Python開發環境設置

在開始構建AI應用之前，我們需要設置一個完整的Python開發環境。這個環節對於初學者來說非常重要，正確的環境設置能避免後續開發中的許多問題。

環境要求檢查：

```
import sys
import subprocess

def check_environment():
    """檢查開發環境是否就緒"""
    print("=== 開發環境檢查 ===")

    # 檢查Python版本
    python_version = sys.version
    print(f"Python版本：{python_version}")

    if sys.version_info < (3, 7):
        print("⚠️  建議使用Python 3.7或更高版本")
    else:
```

```
        print("✅ Python版本符合要求")

    # 檢查必需的庫
    required_packages = ['requests', 'json']
    optional_packages = ['flask', 'streamlit', 'gradio']

    print("\n必需庫檢查：")
    for package in required_packages:
        try:
            __import__(package)
            print(f"✅ {package} 已安裝")
        except ImportError:
            print(f"❌ {package} 未安裝，請運行：pip install {package}")

    print("\n可選庫檢查（用於Web應用）：")
    for package in optional_packages:
        try:
            __import__(package)
            print(f"✅ {package} 已安裝")
        except ImportError:
            print(f"⭕ {package} 未安裝（可選）")

# 運行環境檢查
check_environment()
```

完整的依賴安裝腳本：

```
# requirements.txt
requests>=2.25.0
flask>=2.0.0
streamlit>=1.0.0
python-dotenv>=0.19.0
gradio>=3.0.0

# 安裝命令
pip install -r requirements.txt
```

5.2 基礎聊天機器人開發

讓我們從最簡單的聊天機器人開始，逐步構建功能完整的AI應用。

5.2.1 命令行版本

基礎版本：

```
import requests
import json
import os
from datetime import datetime
```

```
class SimpleChatBot:
    def __init__(self, api_key, model="gpt-4o-mini"):
        self.api_key = api_key
        self.model = model
        self.conversation_history = []
        self.base_url = "https://ai-
apigateway.must.edu.mo/openhub/v1/chat/completions"

    def set_system_prompt(self, prompt):
        """設置系統提示"""
        self.conversation_history = [
            {"role": "system", "content": prompt}
        ]

    def chat(self, user_input):
        """進行對話"""
        # 添加用戶輸入
        self.conversation_history.append({
            "role": "user",
            "content": user_input
        })

        # 準備API請求
        headers = {
            "Authorization": self.api_key,
            "Content-Type": "application/json"
        }

        data = {
            "model": self.model,
            "messages": self.conversation_history,
            "temperature": 0.7,
            "max_tokens": 1000
        }

        try:
            response = requests.post(self.base_url, headers=headers, json=data)

            if response.status_code == 200:
                result = response.json()
                ai_response = result["choices"][0]["message"]["content"]

                # 添加AI回覆到歷史
                self.conversation_history.append({
                    "role": "assistant",
                    "content": ai_response
                })

                return ai_response
            else:
                return f"錯誤：{response.status_code} - {response.text}"

        except Exception as e:
```

```
            return f"請求失敗：{e}"

    def save_conversation(self, filename=None):
        """保存對話歷史"""
        if not filename:
            timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
            filename = f"conversation_{timestamp}.json"

        with open(filename, 'w', encoding='utf-8') as f:
            json.dump(self.conversation_history, f, ensure_ascii=False, indent=2)

        print(f"對話已保存到：{filename}")

    def load_conversation(self, filename):
        """載入對話歷史"""
        try:
            with open(filename, 'r', encoding='utf-8') as f:
                self.conversation_history = json.load(f)
            print(f"對話已從 {filename} 載入")
        except FileNotFoundError:
            print(f"文件 {filename} 不存在")
        except Exception as e:
            print(f"載入失敗：{e}")

def main():
    """主程序"""
    print("=== WeMust AI 聊天機器人 ===")
    print("輸入 'quit' 退出，'save' 保存對話，'load' 載入對話")

    # 初始化機器人
    api_key = input("請輸入你的API密鑰：").strip()
    if not api_key:
        print("API密鑰不能為空！")
        return

    bot = SimpleChatBot("Bearer " + api_key)

    # 設置系統角色
    system_prompt = input("設置AI角色（直接回車使用默認）：").strip()
    if not system_prompt:
        system_prompt = "你是一個友善、有幫助的AI助手"

    bot.set_system_prompt(system_prompt)
    print(f"AI角色已設置：{system_prompt}\n")

    # 開始對話循環
    while True:
        user_input = input("你：").strip()

        if user_input.lower() == 'quit':
            print("再見！")
            break
        elif user_input.lower() == 'save':
            bot.save_conversation()
```

```
            continue
        elif user_input.lower().startswith('load'):
            filename = user_input.split(' ', 1)[1] if ' ' in user_input else
input("請輸入文件名：")
            bot.load_conversation(filename)
            continue
        elif not user_input:
            continue

        print("AI正在思考...")
        response = bot.chat(user_input)
        print(f"AI：{response}\n")

if __name__ == "__main__":
    main()
```

5.2.2 Web版本

使用Flask創建一個簡單的Web聊天界面：

Flask後端（app.py）：

```
from flask import Flask, render_template_string, request, jsonify, session
import requests
import json
import uuid
from datetime import datetime
import os

app = Flask(__name__)
app.secret_key = 'wemust-ai-chat-secret-key-2025'  # 請在生產環境中使用隨機密鑰

class WebChatBot:
    def __init__(self):
        self.base_url = "https://ai-
apigateway.must.edu.mo/openhub/v1/chat/completions"

    def chat(self, api_key, messages, model="gpt-4o-mini", temperature=0.7):
        """發送聊天請求"""
        headers = {
            "Authorization": "Bearer " + api_key,
            "Content-Type": "application/json"
        }

        data = {
            "model": model,
            "messages": messages,
            "temperature": temperature,
            "max_tokens": 1000
        }
```

```
        try:
            response = requests.post(self.base_url, headers=headers, json=data,
timeout=30)

            if response.status_code == 200:
                result = response.json()
                return {
                    "success": True,
                    "content": result["choices"][0]["message"]["content"],
                    "usage": result.get("usage", {})
                }
            else:
                return {
                    "success": False,
                    "error": f"API錯誤：{response.status_code} - {response.text}"
                }

        except requests.exceptions.Timeout:
            return {
                "success": False,
                "error": "請求超時，請稍後再試"
            }
        except requests.exceptions.ConnectionError:
            return {
                "success": False,
                "error": "網絡連接錯誤，請檢查網絡設置"
            }
        except Exception as e:
            return {
                "success": False,
                "error": f"請求失敗：{str(e)}"
            }

# HTML模板直接嵌入到Python代碼中，避免模板文件路徑問題
HTML_TEMPLATE = """
<!DOCTYPE html>
<html lang="zh-TW">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>WeMust AI 聊天機器人</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }

        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            height: 100vh;
            display: flex;
```

```
            justify-content: center;
            align-items: center;
        }

        .chat-container {
            width: 90%;
            max-width: 800px;
            height: 90vh;
            background: white;
            border-radius: 15px;
            box-shadow: 0 20px 40px rgba(0,0,0,0.1);
            display: flex;
            flex-direction: column;
            overflow: hidden;
        }

        .chat-header {
            background: #4a5568;
            color: white;
            padding: 20px;
            text-align: center;
        }

        .chat-messages {
            flex: 1;
            overflow-y: auto;
            padding: 20px;
            background: #f7fafc;
        }

        .message {
            margin-bottom: 15px;
            display: flex;
            align-items: flex-start;
        }

        .message.user {
            justify-content: flex-end;
        }

        .message-content {
            max-width: 70%;
            padding: 12px 16px;
            border-radius: 18px;
            word-wrap: break-word;
            white-space: pre-wrap;
        }

        .message.user .message-content {
            background: #4299e1;
            color: white;
        }

        .message.ai .message-content {
```

```
            background: white;
            border: 1px solid #e2e8f0;
            color: #2d3748;
        }

        .message.error .message-content {
            background: #fed7d7;
            border: 1px solid #fc8181;
            color: #c53030;
        }

        .chat-input-area {
            padding: 20px;
            background: white;
            border-top: 1px solid #e2e8f0;
        }

        .settings-panel {
            display: none;
            padding: 15px;
            background: #edf2f7;
            border-top: 1px solid #e2e8f0;
        }

        .input-group {
            display: flex;
            gap: 10px;
            margin-bottom: 10px;
            flex-wrap: wrap;
        }

        .input-group input, .input-group select {
            padding: 10px;
            border: 1px solid #cbd5e0;
            border-radius: 8px;
            font-size: 14px;
            flex: 1;
            min-width: 150px;
        }

        .message-input {
            flex: 1;
            padding: 12px;
            border: 1px solid #cbd5e0;
            border-radius: 25px;
            outline: none;
            font-size: 16px;
            min-width: 200px;
        }

        .send-button {
            padding: 12px 24px;
            background: #4299e1;
            color: white;
```

```
            border: none;
            border-radius: 25px;
            cursor: pointer;
            font-size: 16px;
            transition: background 0.3s;
            white-space: nowrap;
        }

        .send-button:hover {
            background: #3182ce;
        }

        .send-button:disabled {
            background: #a0aec0;
            cursor: not-allowed;
        }

        .control-buttons {
            display: flex;
            gap: 10px;
            margin-bottom: 10px;
            flex-wrap: wrap;
        }

        .control-button {
            padding: 8px 16px;
            background: #718096;
            color: white;
            border: none;
            border-radius: 6px;
            cursor: pointer;
            font-size: 14px;
            white-space: nowrap;
        }

        .control-button:hover {
            background: #4a5568;
        }

        .loading {
            display: none;
            text-align: center;
            color: #718096;
            font-style: italic;
            margin: 10px 0;
        }

        .status-info {
            font-size: 12px;
            color: #718096;
            text-align: center;
            margin-top: 5px;
        }
```

```
        /* 響應式設計 */
        @media (max-width: 768px) {
            .chat-container {
                width: 95%;
                height: 95vh;
            }

            .input-group {
                flex-direction: column;
            }

            .input-group input, .input-group select {
                min-width: unset;
            }

            .control-buttons {
                justify-content: center;
            }
        }
    </style>
</head>
<body>
    <div class="chat-container">
        <div class="chat-header">
            <h1>🤖 WeMust AI 聊天機器人</h1>
            <p>智能對話助手 - 修復版</p>
        </div>

        <div class="chat-messages" id="chatMessages">
            <div class="message ai">
                <div class="message-content">
                    👋 你好！我是WeMust AI助手，很高興為你服務。

請先在下方設置你的API密鑰，然後我們就可以開始對話了！

💡 使用提示：
• 點擊"設置"按鈕配置API密鑰和AI角色
• 支持多種AI模型選擇
• 可以隨時清除對話或導出記錄
                </div>
            </div>
        </div>

        <div class="settings-panel" id="settingsPanel">
            <div class="input-group">
                <input type="password" id="apiKey" placeholder="請輸入API密鑰"
style="flex: 2;">
                <input type="text" id="systemPrompt" placeholder="設置AI角色（可
選）" style="flex: 2;">
                <select id="modelSelect">
                    <option value="gpt-4o-mini">GPT-4o Mini (推薦)</option>
                    <option value="glm-4-flash">GLM-4 Flash</option>
                    <option value="glm-4">GLM-4</option>
                </select>
```

```
            </div>
            <div class="status-info" id="statusInfo">
                請設置API密鑰後開始對話
            </div>
        </div>

        <div class="chat-input-area">
            <div class="control-buttons">
                <button class="control-button" onclick="toggleSettings()">⚙️ 設置
</button>
                <button class="control-button" onclick="clearChat()">🗑️ 清除對話
</button>
                <button class="control-button" onclick="exportChat()">📥 導出對話
</button>
                <button class="control-button" onclick="showHelp()">❓ 幫助
</button>
            </div>

            <div class="input-group">
                <input type="text" class="message-input" id="messageInput"
                       placeholder="輸入你的消息..."
onkeypress="handleKeyPress(event)">
                <button class="send-button" id="sendButton"
onclick="sendMessage()">發送</button>
            </div>

            <div class="loading" id="loading">🤔 AI正在思考中...</div>
        </div>
    </div>

    <script>
        let isSettingsVisible = false;
        let messageCount = 0;

        function toggleSettings() {
            const panel = document.getElementById('settingsPanel');
            isSettingsVisible = !isSettingsVisible;
            panel.style.display = isSettingsVisible ? 'block' : 'none';

            if (isSettingsVisible) {
                document.getElementById('apiKey').focus();
            }
        }

        function handleKeyPress(event) {
            if (event.key === 'Enter' && !event.shiftKey) {
                event.preventDefault();
                sendMessage();
            }
        }

        async function sendMessage() {
            const messageInput = document.getElementById('messageInput');
            const apiKey = document.getElementById('apiKey').value.trim();
```

```
            const message = messageInput.value.trim();

            if (!apiKey) {
                alert('請先設置API密鑰！');
                toggleSettings();
                return;
            }

            if (!message) {
                alert('請輸入消息！');
                messageInput.focus();
                return;
            }

            // 顯示用戶消息
            addMessage('user', message);
            messageInput.value = '';
            messageCount++;

            // 顯示加載狀態
            setLoading(true);
            updateStatus('正在發送請求...');

            try {
                const response = await fetch('/api/chat', {
                    method: 'POST',
                    headers: {
                        'Content-Type': 'application/json',
                    },
                    body: JSON.stringify({
                        api_key: apiKey,
                        message: message,
                        system_prompt:
document.getElementById('systemPrompt').value,
                        model: document.getElementById('modelSelect').value
                    })
                });

                const result = await response.json();

                if (result.success) {
                    addMessage('ai', result.content);
                    updateStatus(`對話進行中 (${messageCount} 條消息)`);

                    // 顯示使用統計
                    if (result.usage) {
                        console.log('Token使用:', result.usage);
                    }
                } else {
                    addMessage('error', `錯誤：${result.error}`);
                    updateStatus('發送失敗，請檢查設置');
                }

            } catch (error) {
```

```
            addMessage('error', `網絡錯誤：${error.message}`);
            updateStatus('網絡連接失敗');
        } finally {
            setLoading(false);
            messageInput.focus();
        }
    }

    function addMessage(type, content) {
        const messagesContainer = document.getElementById('chatMessages');
        const messageDiv = document.createElement('div');
        messageDiv.className = `message ${type}`;

        const contentDiv = document.createElement('div');
        contentDiv.className = 'message-content';

        // 處理換行
        contentDiv.textContent = content;

        messageDiv.appendChild(contentDiv);
        messagesContainer.appendChild(messageDiv);

        // 滾動到底部
        messagesContainer.scrollTop = messagesContainer.scrollHeight;
    }

    function setLoading(isLoading) {
        const loading = document.getElementById('loading');
        const sendButton = document.getElementById('sendButton');
        const messageInput = document.getElementById('messageInput');

        loading.style.display = isLoading ? 'block' : 'none';
        sendButton.disabled = isLoading;
        messageInput.disabled = isLoading;

        if (isLoading) {
            sendButton.textContent = '發送中...';
        } else {
            sendButton.textContent = '發送';
        }
    }

    function updateStatus(message) {
        const statusInfo = document.getElementById('statusInfo');
        statusInfo.textContent = message;
    }

    async function clearChat() {
        if (confirm('確定要清除所有對話記錄嗎？')) {
            try {
                await fetch('/api/clear', { method: 'POST' });
                document.getElementById('chatMessages').innerHTML = `
                    <div class="message ai">
                        <div class="message-content">
```

```
                                ✨ 對話已清除，我們可以重新開始！有什麼我可以幫助你的
嗎？
                            </div>
                        </div>
                    `;
                    messageCount = 0;
                    updateStatus('對話已清除');
                } catch (error) {
                    alert('清除失敗：' + error.message);
                }
            }
        }

        async function exportChat() {
            try {
                const response = await fetch('/api/export');
                const result = await response.json();

                if (result.success) {
                    const blob = new Blob([result.conversation], { type:
'text/plain;charset=utf-8' });
                    const url = URL.createObjectURL(blob);
                    const a = document.createElement('a');
                    a.href = url;
                    a.download = `WeMust_AI_Chat_${new
Date().toISOString().slice(0,10)}.txt`;
                    document.body.appendChild(a);
                    a.click();
                    document.body.removeChild(a);
                    URL.revokeObjectURL(url);

                    updateStatus('對話已導出');
                } else {
                    alert('導出失敗');
                }
            } catch (error) {
                alert('導出失敗：' + error.message);
            }
        }

        function showHelp() {
            const helpMessage = `
🤖 WeMust AI 聊天機器人使用說明

📋 基本操作：
• 在設置中輸入API密鑰
• 選擇合適的AI模型
• 輸入消息開始對話

⚙️ 功能說明：
• 設置：配置API密鑰和AI角色
• 清除對話：刪除所有對話記錄
• 導出對話：保存對話到文件
• 幫助：顯示此說明
```

```
🔧 快捷鍵：
• Enter：發送消息
• Shift+Enter：換行

💡 使用技巧：
• 可以設置AI的角色和性格
• 支持多種AI模型選擇
• 對話會自動保存在瀏覽器中
            `;

            addMessage('ai', helpMessage);
        }

        // 頁面加載完成後的初始化
        document.addEventListener('DOMContentLoaded', function() {
            updateStatus('請設置API密鑰後開始對話');
            document.getElementById('messageInput').focus();
        });
    </script>
</body>
</html>
"""

bot = WebChatBot()

@app.route('/')
def index():
    """主頁面 - 使用嵌入式HTML模板"""
    if 'session_id' not in session:
        session['session_id'] = str(uuid.uuid4())
        session['messages'] = []
    return render_template_string(HTML_TEMPLATE)

@app.route('/api/chat', methods=['POST'])
def api_chat():
    """聊天API端點"""
    try:
        data = request.get_json()

        # 驗證必需參數
        if not data:
            return jsonify({"success": False, "error": "請求數據為空"})

        if not data.get('api_key'):
            return jsonify({"success": False, "error": "API密鑰不能為空"})

        if not data.get('message'):
            return jsonify({"success": False, "error": "消息不能為空"})

        # 獲取或初始化對話歷史
        if 'messages' not in session:
            session['messages'] = []
```

```
        messages = session['messages'].copy()

        # 如果是新對話或需要設置系統角色
        system_prompt = data.get('system_prompt', '').strip()
        if system_prompt and (not messages or messages[0].get('role') !=
'system'):
            # 如果已有系統消息，先移除
            messages = [msg for msg in messages if msg.get('role') != 'system']
            # 添加新的系統消息到開頭
            messages.insert(0, {
                "role": "system",
                "content": system_prompt
            })

        # 添加用戶消息
        messages.append({
            "role": "user",
            "content": data['message']
        })

        # 調用AI
        result = bot.chat(
            api_key=data['api_key'],
            messages=messages,
            model=data.get('model', 'gpt-4o-mini'),
            temperature=data.get('temperature', 0.7)
        )

        if result['success']:
            # 添加AI回覆到歷史
            messages.append({
                "role": "assistant",
                "content": result['content']
            })

            # 保存到session（限制歷史長度）
            if len(messages) > 50:  # 保留最近25輪對話
                system_msgs = [msg for msg in messages if msg.get('role') ==
'system']
                recent_msgs = messages[-48:]  # 保留最近48條消息
                messages = system_msgs + recent_msgs

            session['messages'] = messages

            return jsonify({
                "success": True,
                "content": result['content'],
                "usage": result.get('usage', {}),
                "message_count": len(messages)
            })
        else:
            return jsonify(result)

    except Exception as e:
```

```
        return jsonify({
            "success": False,
            "error": f"服務器錯誤：{str(e)}"
        })

@app.route('/api/clear', methods=['POST'])
def api_clear():
    """清除對話歷史"""
    try:
        session['messages'] = []
        return jsonify({"success": True, "message": "對話歷史已清除"})
    except Exception as e:
        return jsonify({"success": False, "error": str(e)})

@app.route('/api/export', methods=['GET'])
def api_export():
    """導出對話歷史"""
    try:
        messages = session.get('messages', [])

        # 格式化對話歷史
        formatted_conversation = []
        formatted_conversation.append(f"WeMust AI 對話記錄")
        formatted_conversation.append(f"導出時間：{datetime.now().strftime('%Y-%m-
%d %H:%M:%S')}")
        formatted_conversation.append("=" * 50)
        formatted_conversation.append("")

        for msg in messages:
            if msg['role'] == 'user':
                formatted_conversation.append(f"👤 用戶：{msg['content']}")
                formatted_conversation.append("")
            elif msg['role'] == 'assistant':
                formatted_conversation.append(f"🤖 AI：{msg['content']}")
                formatted_conversation.append("")
            elif msg['role'] == 'system':
                formatted_conversation.append(f"⚙️ 系統：{msg['content']}")
                formatted_conversation.append("")

        return jsonify({
            "success": True,
            "conversation": "\n".join(formatted_conversation),
            "timestamp": datetime.now().isoformat()
        })

    except Exception as e:
        return jsonify({"success": False, "error": str(e)})

@app.route('/health')
def health_check():
    """健康檢查端點"""
    return jsonify({
        "status": "healthy",
        "timestamp": datetime.now().isoformat(),
```

```
        "version": "1.1.0"
    })

if __name__ == '__main__':
    print("🚀 WeMust AI Web聊天應用啟動中...")
    print("")
    print("🌐 訪問地址：http://localhost:5000")
    print("🔧 健康檢查：http://localhost:5000/health")
    print("")

    try:
        app.run(debug=True, host='0.0.0.0', port=5000)
    except KeyboardInterrupt:
        print("\n👋 應用已停止")
    except Exception as e:
        print(f"\n❌ 啟動失敗：{e}")
```

5.3 進階應用開發

5.3.1 智能問答系統

構建一個能夠回答特定領域問題的智能問答系統：

```
import json
import requests
from datetime import datetime
import sqlite3

class IntelligentQASystem:
    def __init__(self, api_key, knowledge_base_path="qa_knowledge.db"):
        self.api_key = api_key
        self.base_url = "https://ai-
apigateway.must.edu.mo/openhub/v1/chat/completions"
        self.knowledge_base_path = knowledge_base_path
        self.init_database()

    def init_database(self):
        """初始化知識庫數據庫"""
        conn = sqlite3.connect(self.knowledge_base_path)
        cursor = conn.cursor()

        cursor.execute('''
            CREATE TABLE IF NOT EXISTS qa_pairs (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                question TEXT NOT NULL,
                answer TEXT NOT NULL,
                category TEXT,
                confidence REAL DEFAULT 0.0,
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            )
```

```
        ''')

        cursor.execute('''
            CREATE TABLE IF NOT EXISTS user_feedback (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                qa_id INTEGER,
                user_question TEXT,
                ai_answer TEXT,
                user_rating INTEGER,
                feedback_text TEXT,
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                FOREIGN KEY (qa_id) REFERENCES qa_pairs (id)
            )
        ''')

        conn.commit()
        conn.close()

    def search_knowledge_base(self, question, limit=3):
        """在知識庫中搜索相關問答"""
        conn = sqlite3.connect(self.knowledge_base_path)
        cursor = conn.cursor()

        # 簡單的關鍵詞匹配搜索
        keywords = question.split()
        search_conditions = []
        params = []

        for keyword in keywords:
            search_conditions.append("(question LIKE ? OR answer LIKE ?)")
            params.extend([f"%{keyword}%", f"%{keyword}%"])

        if search_conditions:
            query = f"""
                SELECT question, answer, category, confidence
                FROM qa_pairs
                WHERE {' OR '.join(search_conditions)}
                ORDER BY confidence DESC
                LIMIT ?
            """
            params.append(limit)

            cursor.execute(query, params)
            results = cursor.fetchall()
        else:
            results = []

        conn.close()
        return results

    def add_qa_pair(self, question, answer, category=None, confidence=0.8):
        """添加問答對到知識庫"""
        conn = sqlite3.connect(self.knowledge_base_path)
        cursor = conn.cursor()
```

```
        cursor.execute('''
            INSERT INTO qa_pairs (question, answer, category, confidence)
            VALUES (?, ?, ?, ?)
        ''', (question, answer, category, confidence))

        conn.commit()
        qa_id = cursor.lastrowid
        conn.close()

        return qa_id

    def answer_question(self, question, use_knowledge_base=True):
        """回答問題"""
        context_info = ""

        # 搜索知識庫
        if use_knowledge_base:
            kb_results = self.search_knowledge_base(question)
            if kb_results:
                context_info = "參考知識庫中的相關信息：\n"
                for i, (kb_q, kb_a, category, confidence) in enumerate(kb_results,
1):
                    context_info += f"{i}. 問題：{kb_q}\n    答案：{kb_a}\n"
                context_info += "\n基於以上信息，請回答用戶的問題。如果知識庫信息不
足，請基於你的知識進行補充。\n\n"

        # 構建系統提示
        system_prompt = f"""
        你是一個專業的問答助手，具有以下特點：
        1. 提供準確、有用的答案
        2. 如果不確定答案，會誠實說明
        3. 盡量提供具體的例子和解釋
        4. 保持友善和專業的語調

        {context_info}
        """

        messages = [
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": question}
        ]

        # 調用AI API
        headers = {
            "Authorization": self.api_key,
            "Content-Type": "application/json"
        }

        data = {
            "model": "gpt-4o-mini",
            "messages": messages,
            "temperature": 0.3,  # 較低的溫度以獲得更一致的答案
            "max_tokens": 1000
```

```
        }

        try:
            response = requests.post(self.base_url, headers=headers, json=data)

            if response.status_code == 200:
                result = response.json()
                ai_answer = result["choices"][0]["message"]["content"]

                # 可選：將新的問答對添加到知識庫
                # self.add_qa_pair(question, ai_answer, confidence=0.6)

                return {
                    "success": True,
                    "answer": ai_answer,
                    "knowledge_base_used": len(kb_results) > 0 if
use_knowledge_base else False,
                    "kb_matches": len(kb_results) if use_knowledge_base else 0
                }
            else:
                return {
                    "success": False,
                    "error": f"API錯誤：{response.status_code}"
                }

        except Exception as e:
            return {
                "success": False,
                "error": f"請求失敗：{str(e)}"
            }

    def collect_feedback(self, question, answer, rating, feedback_text=""):
        """收集用戶反饋"""
        conn = sqlite3.connect(self.knowledge_base_path)
        cursor = conn.cursor()

        cursor.execute('''
            INSERT INTO user_feedback (user_question, ai_answer, user_rating,
feedback_text)
            VALUES (?, ?, ?, ?)
        ''', (question, answer, rating, feedback_text))

        conn.commit()
        conn.close()

        # 如果評分較高，可以考慮將問答對添加到知識庫
        if rating >= 4:
            self.add_qa_pair(question, answer, confidence=rating/5.0)

# 使用示例
def demo_qa_system():
    """演示問答系統"""
    api_key = input("請輸入API密鑰：")
    qa_system = IntelligentQASystem(api_key)
```

```
    # 預填充一些知識庫數據
    sample_qa_pairs = [
        ("什麼是機器學習？", "機器學習是人工智能的一個分支，它使計算機能夠在沒有明確編程的情況下學習和改進。", "AI基礎"),
        ("Python如何安裝庫？", "使用pip命令安裝Python庫，例如：pip install requests", "編程"),
        ("API是什麼？", "API（應用程序編程接口）是不同軟件組件之間通信的接口。", "技術概念")
    ]

    for question, answer, category in sample_qa_pairs:
        qa_system.add_qa_pair(question, answer, category, 0.9)

    print("=== 智能問答系統演示 ===")
    print("輸入 'quit' 退出")

    while True:
        question = input("\n請輸入你的問題：").strip()

        if question.lower() == 'quit':
            break

        if not question:
            continue

        print("正在查找答案...")
        result = qa_system.answer_question(question)

        if result["success"]:
            print(f"\n答案：{result['answer']}")

            if result["knowledge_base_used"]:
                print(f"（參考了知識庫中的 {result['kb_matches']} 條相關信息）")

            # 收集反饋
            try:
                rating = int(input("\n請為這個答案評分（1-5）："))
                if 1 <= rating <= 5:
                    feedback = input("有什麼建議嗎？（可選）：")
                    qa_system.collect_feedback(question, result['answer'], rating, feedback)
                    print("感謝你的反饋！")
            except ValueError:
                pass

        else:
            print(f"錯誤：{result['error']}")

if __name__ == "__main__":
    demo_qa_system()
```

通過這些實戰教程，你已經學會了如何構建實用的AI應用。從簡單的命令行聊天機器人到複雜的Web應用和智能問答系統，這些例子展示了WeMust AI API的強大功能和靈活性。

第六章：完整API參考

6.1 請求格式規範

WeMust AI API遵循RESTful設計原則，使用標準的HTTP方法和JSON格式進行數據交換。理解正確的請求格式是成功調用API的基礎。

基本請求結構：

```
POST https://ai-apigateway.must.edu.mo/openhub/v1/chat/completions
Content-Type: application/json
Authorization: Bearer YOUR_API_KEY

{
  "model": "gpt-4o-mini",
  "messages": [
    {
      "role": "system",
      "content": "你是一個有幫助的助手"
    },
    {
      "role": "user",
      "content": "你好"
    }
  ],
  "temperature": 0.7,
  "max_tokens": 1000
}
```

請求頭部（Headers）詳解：

頭部名稱	必需	說明	示例值
Content-Type	是	請求內容類型	application/json
Authorization	是	API認證密鑰	YOUR_API_KEY
User-Agent	否	客戶端標識	MyApp/1.0

重要注意事項：

- Authorization頭部，需要"Bearer "前綴加上API密鑰，中間使用空格分開
- 所有請求必須使用HTTPS協議
- 請求體必須是有效的JSON格式
- 字符編碼必須是UTF-8

6.2 響應格式說明

API響應統一使用JSON格式，包含請求結果和相關元數據。理解響應結構有助於正確處理API返回的數據。

成功響應結構：

```
{
  "id": "chatcmpl-BJW0amfwix3vcWSYb5eND65BLupmY",
  "object": "chat.completion",
  "created": 1743990972,
  "model": "gpt-4o-mini",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "你好！我是WeMust AI助手，很高興為你服務。有什麼我可以幫助你的嗎？"
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 15,
    "completion_tokens": 25,
    "total_tokens": 40
  }
}
```

響應字段詳解：

字段名	類型	說明
id	string	請求的唯一標識符
object	string	對象類型，固定為"chat.completion"
created	integer	創建時間戳（Unix時間戳）
model	string	實際使用的模型名稱
choices	array	生成的回覆選項數組
choices[].index	integer	選項索引
choices[].message	object	生成的消息對象
choices[].message.role	string	消息角色，固定為"assistant"
choices[].message.content	string	生成的文本內容
choices[].finish_reason	string	生成停止的原因
usage	object	Token使用統計
usage.prompt_tokens	integer	輸入使用的Token數

字段名	類型	說明
usage.completion_tokens	integer	輸出使用的Token數
usage.total_tokens	integer	總Token使用數

finish_reason可能的值：

值	說明
stop	自然結束
length	達到最大長度限制
content_filter	內容被過濾
tool_calls	需要調用工具

流式響應格式：

```
data: {"id":"chatcmpl-
123","object":"chat.completion.chunk","created":1694268190,"model":"gpt-4o-
mini","choices":[{"index":0,"delta":
{"role":"assistant","content":""},"finish_reason":null}]}

data: {"id":"chatcmpl-
123","object":"chat.completion.chunk","created":1694268190,"model":"gpt-4o-
mini","choices":[{"index":0,"delta":{"content":"你好"},"finish_reason":null}]}

data: {"id":"chatcmpl-
123","object":"chat.completion.chunk","created":1694268190,"model":"gpt-4o-
mini","choices":[{"index":0,"delta":{},"finish_reason":"stop"}]}

data: [DONE]
```

6.3 錯誤碼完整列表

當API調用失敗時，會返回相應的HTTP狀態碼和錯誤信息。了解這些錯誤碼有助於快速診斷和解決問題。

錯誤響應格式：

```
{
  "error": {
    "message": "您的API密鑰無效",
    "type": "authentication_error",
    "param": null,
    "code": "invalid_api_key"
  }
}
```

HTTP狀態碼對照表：

狀態碼	錯誤類型	常見原因	解決方法
400	invalid_request_error	請求參數錯誤	檢查請求格式和參數
401	authentication_error	認證失敗	檢查API密鑰是否正確
403	permission_error	權限不足	檢查API密鑰權限設置
404	not_found_error	資源不存在	檢查API端點URL
429	rate_limit_error	請求頻率超限	降低請求頻率或升級配額
500	api_error	服務器內部錯誤	稍後重試或聯繫技術支持
502	service_unavailable	服務不可用	檢查服務狀態，稍後重試
503	service_unavailable	服務暫時不可用	等待服務恢復

詳細錯誤代碼說明：

認證相關錯誤：

- `invalid_api_key`: API密鑰無效或格式錯誤
- `missing_api_key`: 缺少API密鑰
- `expired_api_key`: API密鑰已過期
- `suspended_api_key`: API密鑰已被暫停

請求參數錯誤：

- `invalid_model`: 模型名稱無效
- `invalid_messages`: 消息格式錯誤
- `invalid_temperature`: 溫度參數超出範圍
- `invalid_max_tokens`: max_tokens參數無效
- `missing_required_parameter`: 缺少必需參數

配額和限制錯誤：

- `rate_limit_exceeded`: 超出請求頻率限制
- `quota_exceeded`: 超出使用配額
- `token_limit_exceeded`: 單次請求Token數超限

服務相關錯誤：

- `model_overloaded`: 模型負載過高
- `service_maintenance`: 服務維護中
- `internal_error`: 內部服務錯誤

錯誤處理最佳實踐：

```
import requests
import time
import json
```

```python
def robust_api_call(api_key, messages, max_retries=3, retry_delay=1):
    """
    帶錯誤處理和重試機制的API調用
    """
    headers = {
        "Authorization": api_key,
        "Content-Type": "application/json"
    }

    data = {
        "model": "gpt-4o-mini",
        "messages": messages,
        "temperature": 0.7,
        "max_tokens": 1000
    }

    for attempt in range(max_retries):
        try:
            response = requests.post(
                "https://ai-apigateway.must.edu.mo/openhub/v1/chat/completions",
                headers=headers,
                json=data,
                timeout=30
            )

            # 成功響應
            if response.status_code == 200:
                return {
                    "success": True,
                    "data": response.json()
                }

            # 解析錯誤響應
            try:
                error_data = response.json()
                error_info = error_data.get("error", {})
            except:
                error_info = {"message": response.text}

            # 根據錯誤類型決定是否重試
            if response.status_code in [429, 500, 502, 503]:
                # 可重試的錯誤
                if attempt < max_retries - 1:
                    wait_time = retry_delay * (2 ** attempt)  # 指數退避
                    print(f"遇到可重試錯誤，{wait_time}秒後重試...")
                    time.sleep(wait_time)
                    continue

            # 不可重試的錯誤或重試次數用盡
            return {
                "success": False,
                "error": {
                    "status_code": response.status_code,
```

```
                    "message": error_info.get("message", "未知錯誤"),
                    "type": error_info.get("type", "unknown_error"),
                    "code": error_info.get("code", "unknown")
                }
            }

        except requests.exceptions.Timeout:
            if attempt < max_retries - 1:
                print(f"請求超時，{retry_delay}秒後重試...")
                time.sleep(retry_delay)
                continue
            return {
                "success": False,
                "error": {
                    "message": "請求超時",
                    "type": "timeout_error"
                }
            }

        except requests.exceptions.ConnectionError:
            if attempt < max_retries - 1:
                print(f"連接錯誤，{retry_delay}秒後重試...")
                time.sleep(retry_delay)
                continue
            return {
                "success": False,
                "error": {
                    "message": "網絡連接錯誤",
                    "type": "connection_error"
                }
            }

        except Exception as e:
            return {
                "success": False,
                "error": {
                    "message": f"未預期的錯誤：{str(e)}",
                    "type": "unexpected_error"
                }
            }

    return {
        "success": False,
        "error": {
            "message": "重試次數用盡",
            "type": "max_retries_exceeded"
        }
    }

# 使用示例
def handle_api_response(result):
    """處理API響應結果"""
    if result["success"]:
        data = result["data"]
```

```
            content = data["choices"][0]["message"]["content"]
            usage = data.get("usage", {})

            print(f"AI回覆：{content}")
            print(f"Token使用：{usage.get('total_tokens', 0)}")

        else:
            error = result["error"]
            error_type = error.get("type", "unknown")

            # 根據錯誤類型提供具體的解決建議
            if error_type == "authentication_error":
                print("❌ 認證錯誤：請檢查API密鑰是否正確")
            elif error_type == "rate_limit_error":
                print("⏰ 請求頻率超限：請稍後再試或升級配額")
            elif error_type == "invalid_request_error":
                print("📝 請求參數錯誤：請檢查請求格式")
            elif error_type == "quota_exceeded":
                print("💰 配額不足：請檢查賬戶餘額或升級套餐")
            else:
                print(f"❌ 錯誤：{error.get('message', '未知錯誤')}")

# 測試錯誤處理
messages = [
    {"role": "user", "content": "你好"}
]

result = robust_api_call("your_api_key", messages)
handle_api_response(result)
```

6.4 模型能力對比表

不同的AI模型具有不同的特點和適用場景，選擇合適的模型對於獲得最佳效果和控制成本都很重要。

模型詳細對比表：

模型名稱	上下文長度	主要特點	適用場景	相對成本	響應速度
gpt-5	272K	深度思考，思考總結，多模態	通用、複雜任務、工具調用	★★★	★★
gpt-5-mini	272K	高性價比，快速響應，多模態	日常對話、簡單任務	★★	★★★
gpt-5-nano	272K	快速響應，多模態	簡單任務	★	★★★★
gpt-4o-mini	128K	高性價比，快速響應	日常對話、簡單任務	★	★★★★★

模型名稱	上下文長度	主要特點	適用場景	相對成本	響應速度
gpt-4o-2024-08-06	128K	平衡 性能	通用任務 、內容創作	★★★★	★★★★★
gpt-4-turbo	128K	高性能推理	複雜分析、專業任務	★★★	★★★
gpt-4-32k	32K	長文本處理	文檔分析、長對話	★★★	★★★
gpt-4.1	32K	最新版本	最新功能測試	★★★	★★★
gpt-4.1-mini	32K	輕量級最新版	高效日常使用	★★	★★★★
gpt-4.1-nano	32K	超輕量級	簡單快速任務	★	★★★★★
glm-4	128K	中文優化	中文內容處理	★★	★★★★
glm-4-air	128K	輕量中文模型	中文日常對話	★	★★★★★
glm-4-airx	128K	增強中文模型	中文專業任務	★★	★★★★
glm-4-flash	128K	快速中文模型	中文快速響應	★	★★★★★
glm-4v-plus	128K	多模態模型	圖文理解	★★★	★★★
glm-4-4v	128K	視覺理解	圖像分析	★★★	★★★
glm-4.5	128K	數學與邏輯、情感理解、長文寫作、代碼編寫、前端製作	複雜任務	★★★★	★★★★
glm-4.5-x	128K	GLM-4.5快速版	高性能、强推理、極速相應	需要快速響應的相對複雜任務	★★★
glm-4.5-air	128K	高性價比、輕量級、強性能	快速響應、輕量應用	★★	★★★★
glm-4.5-airx	128K	輕量級、強性能、極速響應	極速響應、超輕量應用	★	★★★★★
glm-4.5-flash	128K	免費、高效、多功能	通用任務	無成本	★★★
deepseek-r1	128K	推理專用	數學、邏輯推理	★★	★★★

- gpt-5系列模型不支持：temperature、top_p、presence_penalty、frequency_penalty、logprobs、top_logprobs、logit_bias、max_tokens等参数。详见：https://learn.microsoft.com/zh-tw/azure/ai-foundry/openai/how-to/reasoning?tabs=gpt-5%2Cpython-secure%2Cpy
- gpt-5系列中的max_tokens參數需要替換為max_completion_tokens，其餘模型仍使用max_tokens

模型選擇決策樹：

```
開始
├── 需要處理圖像？
│   ├── 是 → glm-4v-plus 或 glm-4-4v
│   └── 否 → 繼續
├── 主要使用中文？
│   ├── 是
│   │   ├── 需要高性能？ → glm-4 或 glm-4-airx
│   │   └── 日常使用？ → glm-4-air 或 glm-4-flash
│   └── 否 → 繼續
├── 需要數學推理？
│   ├── 是 → deepseek-r1
│   └── 否 → 繼續
├── 預算有限？
│   ├── 是 → gpt-4o-mini 或 gpt-4.1-nano
│   └── 否 → 繼續
├── 需要最高性能？
│   ├── 是 → gpt-4-turbo 或 gpt-4.1
│   └── 否 → gpt-4o（推薦）
```

性能測試對比：

```python
import time
import requests

def benchmark_models():
    """對比不同模型的性能"""
    models_to_test = [
        "gpt-4o-mini",
        "glm-4-flash",
        "glm-4"
    ]

    test_prompt = "請用100字左右解釋什麼是人工智能"

    results = {}

    for model in models_to_test:
        print(f"測試模型：{model}")

        start_time = time.time()

        # 這裡調用API（需要實際的API密鑰）
```

```
        # result = call_api(model, test_prompt)

        end_time = time.time()
        response_time = end_time - start_time

        results[model] = {
            "response_time": response_time,
            # "content_quality": evaluate_quality(result),
            # "token_usage": result.get("usage", {})
        }

        print(f"響應時間：{response_time:.2f}秒")
        time.sleep(1)  # 避免頻率限制

    return results

# 模型推薦函數
def recommend_model(use_case, budget_level="medium", language="mixed"):
    """
    根據使用場景推薦模型

    Args:
        use_case: 使用場景 ("chat", "analysis", "creative", "coding", "math")
        budget_level: 預算水平 ("low", "medium", "high")
        language: 主要語言 ("chinese", "english", "mixed")
    """

    recommendations = {
        "chat": {
            "low": {
                "chinese": "glm-4-flash",
                "english": "gpt-4o-mini",
                "mixed": "gpt-4o-mini"
            },
            "medium": {
                "chinese": "glm-4-air",
                "english": "gpt-4o",
                "mixed": "gpt-4o"
            },
            "high": {
                "chinese": "glm-4",
                "english": "gpt-4-turbo",
                "mixed": "gpt-4-turbo"
            }
        },
        "analysis": {
            "low": {
                "chinese": "glm-4",
                "english": "gpt-4o",
                "mixed": "gpt-4o"
            },
            "medium": {
                "chinese": "glm-4",
                "english": "gpt-4-turbo",
```

```
                "mixed": "gpt-4-turbo"
            },
            "high": {
                "chinese": "glm-4",
                "english": "gpt-4-turbo",
                "mixed": "gpt-4-turbo"
            }
        },
        "math": {
            "low": "deepseek-r1",
            "medium": "deepseek-r1",
            "high": "gpt-4-turbo"
        }
    }

    if use_case == "math":
        return recommendations[use_case][budget_level]
    else:
        return recommendations[use_case][budget_level][language]

# 使用示例
print("聊天場景推薦：", recommend_model("chat", "medium", "chinese"))
print("分析場景推薦：", recommend_model("analysis", "low", "english"))
print("數學場景推薦：", recommend_model("math", "high"))
```

通過這個完整的API參考，你現在擁有了使用WeMust AI API所需的所有技術細節。無論是處理錯誤、選擇模型還是優化性能，這些信息都將幫助你構建更穩定和高效的AI應用。

附錄

A. 常見問題解答（FAQ）

A.1 認證和API密鑰相關

Q: 如何獲取API密鑰？ A: 訪問WeMust AI平台（https://openhub-web.must.edu.mo/api-apply），註冊賬號後在控制台的"API密鑰"頁面創建新密鑰。

Q: API密鑰有使用期限嗎？ A: API密鑰本身沒有固定期限，但可能會因為安全原因或賬戶狀態變化而失效。建議定期檢查密鑰狀態。

Q: 可以同時使用多個API密鑰嗎？ A: 可以。你可以為不同的項目或環境創建不同的API密鑰，便於管理和監控使用情況。

Q: API密鑰洩露了怎麼辦？ A: 立即在控制台中刪除洩露的密鑰，並創建新的密鑰。同時檢查是否有異常使用記錄。

A.2 技術使用相關

Q: 為什麼收到401錯誤？ A: 401錯誤通常表示認證失敗，請檢查：

- API密鑰是否正確複製（注意前後空格）

- 是否在Authorization頭部正確設置
- 密鑰是否已過期或被禁用

Q: 如何處理中文亂碼問題？ A: 確保：

- 請求和響應都使用UTF-8編碼
- Python文件保存為UTF-8格式
- 在文件開頭添加：`# -*- coding: utf-8 -*-`

Q: API調用很慢怎麼辦？ A: 可以嘗試：

- 使用更快的模型（如gpt-4o-mini）
- 減少max_tokens參數
- 使用流式響應改善用戶體驗
- 檢查網絡連接狀況

Q: 如何控制API使用成本？ A: 建議：

- 選擇合適的模型（不一定要最貴的）
- 合理設置max_tokens限制
- 使用緩存避免重複請求
- 監控使用量和費用

A.3 開發相關

Q: 可以在前端直接調用API嗎？ A: 不建議。API密鑰會暴露給用戶，存在安全風險。建議通過後端服務調用API。

Q: 如何實現對話記憶功能？ A: 在每次請求中包含完整的對話歷史，但要注意：

- 控制歷史長度避免超出token限制
- 可以保留重要的系統消息
- 考慮使用數據庫持久化存儲

Q: 支持哪些編程語言？ A: API是基於HTTP的，支持所有能發送HTTP請求的編程語言，包括Python、JavaScript、Java、C#、PHP等。

Q: 如何處理超長文本？ A: 可以：

- 分段處理長文本
- 使用支持更長上下文的模型
- 提取關鍵信息後再處理
- 使用文本摘要技術

A.4 錯誤處理相關

Q: 遇到429錯誤怎麼辦？ A: 429表示請求頻率超限，可以：

- 降低請求頻率
- 實現指數退避重試機制
- 升級到更高的配額套餐

- 使用請求隊列管理

Q: 如何實現自動重試？ A: 示例代碼：

```
import time
import random

def retry_with_backoff(func, max_retries=3):
    for attempt in range(max_retries):
        try:
            return func()
        except Exception as e:
            if attempt == max_retries - 1:
                raise e
            wait_time = (2 ** attempt) + random.uniform(0, 1)
            time.sleep(wait_time)
```

結語

恭喜你完成了WeMust AI API的完整學習！通過這份文檔，你已經掌握了從基礎概念到高級應用的全套技能。

你現在能夠：

- 理解AI API的工作原理和基本概念
- 正確配置開發環境和認證機制
- 實現各種類型的AI應用，從簡單聊天到複雜的智能系統
- 優化參數以獲得最佳效果
- 處理各種錯誤情況和異常
- 選擇合適的模型和配置

下一步建議：

1. **實踐項目**：選擇一個感興趣的項目開始實際開發
2. **深入學習**：探索更高級的AI應用場景
3. **社區參與**：加入開發者社區，分享經驗和學習
4. **持續更新**：關注API的更新和新功能

文檔信息：

- 版本：v1.1.1
- 作者：ITDO WeMust AI Team
- 最後更新：2025年7月24日
- 許可：教育使用許可